DAyLI – Calendar with AI Chatbot Assistant
Final Report

TU856/4
BSc in Computer Science

Norbert Krupa

C21518659

Supervisor - Paul Bourke

School of Computer Science

Technological University, Dublin

28/03/2025

Abstract

Effectively managing personal and professional tasks and events is something that many users strive from. However, many solutions that exist today are reliant on manual input, and for the user to make sense of their own schedule. With more intelligent AI being developed every year, it's surprising that existing applications have not implemented a native AI powered assistant. These existing applications can be linked with existing assistants like, Amazon Alexa or Siri, but the integration is limited.

This project aimed to address that gap by developing a web-based calendar and task management application with an integrated AI chatbot assistant. The application offered users a hybrid approach to managing their schedules, allowing them to either perform actions manually or interact with the chatbot using natural language. For example, users could schedule an appointment by filling out a form or by simply typing a message to the chatbot.

The application was built using a Django backend, a React frontend, and a PostgreSQL database. The chatbot logic was implemented using the Botpress framework and powered by an OpenAI GPT model. Core features of the system included user authentication, basic CRUD operations for tasks and events, and interactive chatbot functionality. In addition to managing calendar entries, the chatbot was capable of summarising upcoming events, suggesting rescheduling options, and supporting a range of natural-language interactions related to task management.

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Norbert Krupa

Norbert Krupa

28/03/2025

Acknowledgements

Paul Bourke – Supervisor
Jack O’Neill – Final Year Project Coordinator

Table of Contents

1. Introduction

1.1. Project Background

Managing both personal and professional tasks has become increasingly important in recent years. On average, individuals handle around 12 tasks per day, and attempting to manage these manually is often inefficient and time-consuming.

Existing task management tools, such as Google Calendar and Todoist, have improved how users organize their day. These tools, however, rely primarily on manual input and occasionally offer basic voice commands. While they are functional, managing numerous tasks or making frequent changes can still be tedious. Some applications allow integration with personal assistants like Google Assistant or Siri, but these integrations are typically limited and require additional configuration.

This project aimed to address this gap by developing a web-based calendar and task management application with a built-in AI chatbot assistant that had direct access to the user's schedule. The application supported both traditional manual input and natural language interaction via a chatbot. For example, a user could either open the calendar and manually enter appointment details or simply type, "Schedule a dentist appointment for next Tuesday at 3 PM," and the chatbot would handle the rest.

By combining conventional task management with conversational AI, the project sought to create a user-friendly tool suitable for both personal and professional users, regardless of their technical background.

1.2. Project Description

The goal of the project was to develop a hybrid task and calendar management web application integrated with an AI chatbot assistant. Users were able to manage their schedules either through a graphical user interface or via natural language prompts submitted to the chatbot.

The core functionalities of the application included the ability to create, read, update, and delete tasks and events. The chatbot, powered by an OpenAI GPT model, interpreted user prompts and performed the corresponding operations using backend APIs. For instance, when a user typed, "Add a dentist appointment next Tuesday at 3 PM," the chatbot would extract the relevant data, call the appropriate API, and confirm the action with a natural response.

The application was built using:

- A Django backend,
- A React frontend,
- A PostgreSQL database,
- Botpress for chatbot logic and integration.

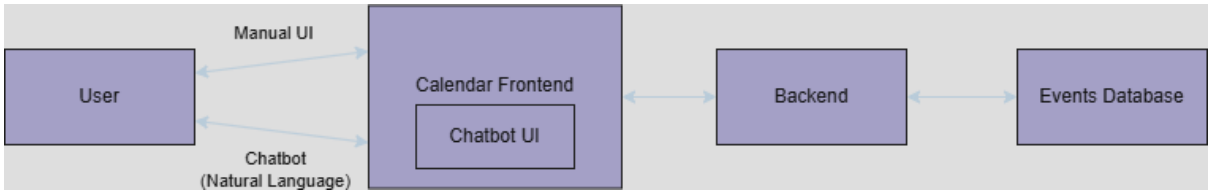


Figure 1.1 – High level system overview

Figure 1.1 illustrates the primary user interactions and system components, showing how the manual UI and chatbot interface communicate with the backend and events database.

1.3. Project Aims and Objectives

Project Aim:
To design and develop a web-based calendar and task management application that integrates a conversational AI chatbot, enabling users to manage their schedules via both traditional interfaces and natural language prompts.

Project Objectives:

- Investigate existing task management tools and AI chatbot frameworks to identify best practices.
- Implement a backend connected to a database to handle user data and tasks/events.
- Develop RESTful APIs to support both manual and chatbot interactions.
- Build a user-friendly frontend interface.
- Integrate an AI-powered chatbot capable of interacting with the backend APIs.
- Enable CRUD (Create, Read, Update, Delete) operations through both the manual interface and chatbot.
- Explore optional enhancements, such as reminders, task prioritization, and grouping.

1.4. Project Scope

In Scope:

- Core task and calendar management features: create, read, update, and delete operations.
- Integration of a built-in AI chatbot capable of interacting with backend APIs without additional configuration.
- Basic user authentication functionality, including login and logout.

Out of Scope:

- Advanced AI features such as sentiment analysis or continuous learning.
- Enterprise-level features, including team collaboration or project management tools.
- Multi-language support – English was the only supported language.

1.5. Thesis Roadmap

This thesis is structured as follows:

- Chapter 2 – Literature Review: Reviews related technologies, tools, and research.
- Chapter 3 – Software Design: Describes the architectural and technical design of the system.
- Chapter 4 – Software Development: Details the development process and key components.
- Chapter 5 – Testing and Evaluation: Discusses testing strategies and evaluation of the final application.
- Chapter 6 – Conclusions and Future Work: Summarizes key findings, outlines potential future enhancements, and briefly discusses the personal journey.

2. Literature Review

2.1. Introduction

The literature review section will create a base for this project by exploring existing tools, technologies, and research related to task management systems and AI chatbots. Task management applications like Google Calendar, Todoist, and Microsoft To Do have become essential tools for personal and professional productivity, but they often rely heavily on manual input or provide limited AI integration. At the same time, developments in natural language processing (NLP) and conversational AI have allowed for the creation of chatbots that can handle complex user interactions, offering new opportunities to improve usability and efficiency in task management.

This chapter examines the landscape of current task management tools and chatbot technologies, analyzing their features, limitations, and relevance to this project. It also explores the state-of-the-art technologies, such as Django, PostgreSQL, and OpenAI GPT models, that enable the integration of conversational AI with traditional interfaces. Additionally, comparisons are made with similar projects and solutions to identify gaps that this project aims to address.

By reviewing existing solutions and relevant literature, this chapter provides a comprehensive understanding of the context for this project and ensures that its design and development are informed by the best practices and advancements in the field. This review also highlights how combining task management systems with conversational AI can meet the needs of diverse users more effectively.

2.2. Alternative Existing Solutions

1. Google Calendar

Google Calendar is one of the most popular task management tools, recognized for its user-friendly interface and integration within Google Workspace. It has features for event scheduling, such as reminders, color-coded calendars, and the ability to invite participants, making it a versatile tool for both personal and professional use. Users can manually create and manage events with a wide range of customizable options, including recurring schedules, location tagging, and notifications.

A notable feature of Google Calendar is its integration with Google Assistant, enabling basic voice commands such as "Schedule a meeting at 10 AM tomorrow". This feature simplifies task creation, making it accessible for users on-the-go. However, the AI capabilities remain quite basic, limited to executing surface-level command. For example, it cannot suggest optimal meeting times based the user's current events or propose adjustments based on detected conflicts.

Additionally, Google Calendar has a section specifically for tasks. This is a very simple checklist where a user can add a task, give it a title, details, time and date, and make it a repeating task. Users can also group tasks into lists. This task section is separate from the calendar but the tasks are displayed in the calendar.

This combined approach makes Google calendar a very useful tool for managing both events and tasks.

2. Microsoft To Do

Microsoft To Do is a lightweight task management app developed to help users to organize their daily tasks, prioritize them, and set reminders. It's one of the core applications of Microsoft 365, allowing for synchronization across multiple devices and integration with apps such as Outlook and Teams. This allows users to turn emails into tasks or schedule tasks within their calendars, creating a productivity workflow.

To Do's user-friendly interface is one of its standout features, designed with simplicity to cater to casual users. With options for grouping tasks, adding notes, and categorizing tasks, Microsoft To Do provides the essential functionalities for task management. The checklist-style organization makes it a good tool for people looking for a simple solution to manage daily tasks.

Microsoft To Do also integrates with Cortana, Microsoft's AI assistant, allowing users to create tasks and reminders using voice commands. This integration is basic and limited to single-step interactions like "Add a task to my To-Do list." Unlike more advanced chatbots that use Natural Language Processing, the system can't understand complex requests or dynamically interact with users to provide suggestion. For example, it can't reorganize tasks based on the user's priorities.

3. Todoist

Todoist is another task management application that stands out for its simplicity and focus on productivity. It was designed to be used by both individuals and teams, offering a large suite of tools for task prioritization and project management. The app's minimalist and intuitive design, makes it accessible to users with any amount of technical experience.

One of its features is the ability to use natural language for task creation. Users can input commands like "Submit weekly log every Friday" which Todoist interprets and schedules accordingly. This works by looking for keywords to understand what the task is called, when it's due, if it should repeat, etc. This feature reduces the need for manual input and reduces the time a user spends on task creation. While Todoist uses some NLP for task management, its reliance on external voice assistants like Google Assistant or Amazon Alexa for voice interaction limits its conversational AI capabilities.

Another strength are its advanced task management features, like recurring tasks and shared projects. These features allow users to create detailed and organized to-do lists tailored to them. Integration with external tools like Google Calendar, Slack, and Microsoft Teams enhances its utility even more, allowing for effective collaboration and time management. Todoist also supports notifications and reminders which can help users to keep up with their tasks.

Although Todoist excels at task organization, it lacks a unified interface for combining event scheduling and task management. Unlike tools specifically designed for calendar integration, Todoist requires users to rely on external applications or third-party plugins to manage events. This separation may create inefficiencies for users seeking an all-in-one solution for both tasks and scheduling.

In summary, Todoist is highly effective for organizing tasks and managing projects, particularly for users who prioritize customization and advanced task features. However, the lack of fully conversational AI and native event scheduling capabilities highlights an opportunity for more integrated solutions that address the evolving needs of modern productivity tools.

2.2.1. Nielson's Heuristics

Heuristic	Google Calendar	Microsoft ToDo	Todoist
1. Visibility of System Status	9 Google Calendar provides clear notifications and progress updates, such as "event added" confirmations.	8 Microsoft To Do shows status updates for sync and reminders but could provide more detailed progress feedback.	9 Todoist provides status updates and notifications effectively, especially for task completion and due dates.
2. Match Between System and Real World	8 Google Calendar's time-based structure is intuitive but may be complex for users unfamiliar with calendar interfaces.	9 Microsoft To Do's to-do list format is straightforward, with familiar language and visual cues.	9 Todoist is task-focused and intuitive, using real-world list and project concepts that are easy to understand.
3. User Control and Freedom	9 Google Calendar offers undo/redo for events and easy navigation, though some settings can be buried.	7 Microsoft To Do lacks extensive undo options, which can limit control for users.	8 Todoist allows users to undo recent actions, edit tasks easily, and navigate back, giving users better freedom.
4. Consistency and Standards	9 Google Calendar follows consistent Google design standards, especially across devices.	8 Microsoft To Do is consistent within itself, but slight UI differences exist between platforms.	9 Todoist has a uniform design and behaves consistently across devices, providing a cohesive experience.
5. Error Prevention	8 Google Calendar prompts for confirmation when deleting events and provides warnings for overlapping schedules.	6 Microsoft To Do's error prevention is basic and lacks confirmation prompts for some actions.	9 Todoist effectively prevents errors with confirmation prompts, and reminders reduce chances of missed tasks.
6. Recognition Rather than Recall	9 Google Calendar has recognizable icons and provides suggestions for previous event names and locations.	8 Microsoft To Do suggests recent tasks and reminders but could improve with more predictive text.	9 Todoist's task templates, recent projects, and intuitive icons support recognition over recall.

7. Flexibility and Efficiency of Use	8 Google Calendar offers keyboard shortcuts and custom views but has limited customization for advanced workflows.	7 Microsoft To Do is simple, ideal for beginners but less flexible for power users.	9 Todoist provides keyboard shortcuts, filters, and customizable views for both basic and advanced users.
8. Aesthetic and Minimalist Design	8 Google Calendar's interface is clean but may feel busy when many events are scheduled.	9 Microsoft To Do has a minimalist design, focused and visually simple, ideal for managing tasks.	9 Todoist has a clean, organized design with optional elements for those who want more detail.
9. Help Users Recognize, Diagnose, and Recover from Errors	7 Google Calendar provides simple error messages but limited guidance on resolution steps.	7 Microsoft To Do's error feedback is basic, sometimes lacking detail on resolution.	9 Todoist's error messages are clear and offer solutions or guidance on fixing issues.
10. Help and Documentation	7 Google Calendar has online support and community forums, though it lacks in-app guidance.	8 Microsoft To Do provides some help links in-app and has extensive online resources.	8 Todoist has both in-app tooltips and online help resources, making it easy to find support.
Total	82	77	88

Based on the heuristic evaluation, Todoist scored the highest overall (88/100), particularly excelling in error prevention, user recognition, and flexibility. However, its lack of native event scheduling and conversational AI remains a limitation that this project aimed to address.

2.3. Technologies researched

Frontend Technologies

1. Angular.js

Angular.js is a powerful framework for creating dynamic web applications. A main feature of Angular is two-way data binding. Two-way data binding allows the model and view to communicate seamlessly. Any changes in the user interface are automatically translated to the data model. The same can be said for the other way around – changes in the data model are automatically reflected in the user interface. This reduces the need for manipulating the Document Object Model (DOM) [4], [5].

Angular.js uses a declarative approach when creating the user interface with HTML. It also follows the Model View Controller (MVC) architecture, which means that there is a separation of concerns between the application logic and user interface. This makes the application easier to maintain [4], [5].

Angular.js features dependency injection. This makes the process of integrating and testing components easier. Dependency injection allows developers to integrate services directly into components, reducing the need for boilerplate code. This makes the application design more modular and reusable. Angular overall promotes a modular structure which allows developers to separate distinct modules in their applications. These modules can then be developed and maintain separately from others. This makes Angular a solid choice for large scale applications [2], [5].

Angular also has a lot of supporting tools like Angular Command Line Interface (CLI) and Angular Material. CLI provides developers with commands for easy setup and management, while Angular Material provides a library of Google Material Design guideline compliant user interface components. This allows developers to quickly create and manage consistent and responsive user interfaces with relative ease [2], [5].

However, Angular.js is not the most beginner friendly frontend framework. It has a steep learning curve because developers need to understand the idea of things like directives, services, and custom components before being able to create user interfaces with the aforementioned ease [4].

Also, Angular.js has an opinionated approach. This means that there is practically one accepted way of doing things in Angular. This makes the framework less flexible, limiting its use in applications that require an unconventional approach [4].

In conclusion, Angular.js is a good choice for complex, large scale applications. The modular architecture, support tools, and high-performance play very well towards apps go beyond a simple application. Angular's focus on modularity in particular makes it suitable for enterprise level applications where scalability is a valued trait [2], [4], [5].

2. React

React is a very popular JavaScript library which is used to build fast, dynamic and scalable user interfaces. It is mainly used for single-page applications. React is component-based. This means that each component which represents a piece of the user interface, should be reusable elsewhere in the application [1], [3].

React is known for its virtual Document Object Model (DOM). Instead of manipulating the browser's DOM, React updates its virtual DOM first. The virtual DOM finds differences between the previous and current states of the user interface and then updates only the necessary parts of the browser's DOM. This improves the applications performance [1], [3].

React has a unidirectional data flow. Data moves in a single direction from parent to child components. This makes the application more predictable and therefore easier to debug. This is useful when managing state transitions [1], [4].

Another feature of React is JavaScriptXML (JSX). This is a syntax extension that combines HTML tags with JavaScript code and logic. This allows developers to define components in a readable way, making the codebase for the frontend easier to understand [3].

React flexibility is another one of its strengths. It can integrate seamlessly with other frameworks and libraries. This makes it suitable for a lot of applications, ranging from simple single page applications to more complex systems. For example, React is commonly used with Next.js for server-side rendering and static site generation. React Native is also used to use React when developing mobile applications [1], [4].

React also has a moderate learning curve. It's popularity means that it has a lot of documentation, tutorials, and a large community of developers willing to help. This makes React accessible to beginners. Its versatility makes it very popular and it has been used to develop a wide range of applications, including social media platforms and e-commerce websites [1], [3].

In conclusion, React is very flexible and performance driven. It changed how user interfaces are developed. The virtual DOM, component-based architecture, readability, and large amount of support make React the preferred choice of many developers [1], [3], [4].

3. Vue.js

Vue.js is a progressive JavaScript framework whose focus is on simplicity and user experience. Being a progressive framework means that developers can use its features incrementally. This makes Vue well suited for both small scale and complex applications. It's seen by many developers a "sweet spot" between Angular and React, combining the features of both frameworks and having a moderate learning curve [2], [4].

Vue is known for its straightforward syntax and declarative approach. This makes creating and managing user interfaces and application logic very intuitive for the developer. Like Angular, Vue uses two-way data binding, synchronizing the model and view. The same advantages apply here, when the data model is updated the user interfaces is automatically updated as well, and vice versa [2], [4].

Vue also has a component-based architecture. Like in React, components can be developed and maintained independently from one another. Vue is most suitable for small to medium scale applications, but its modular design approach also makes it a viable choice for large applications [4].

To further enhance development, Vue has a large number of tools, including a CLI. The CLI can be used for setting up and managing the application, as well as adding plugins. Other tools include Vue Router, to handle page routing, and Vuex, which is used for managing the application state [4].

Despite its advantages, Vue falls behind in popularity compared to Angular and React, especially at an enterprise level. It still has a large community and regular updates which allow it to remain competitive as modern web development framework. Vue's simplicity also makes it more accessible to beginners while also having more advanced capabilities for more complex applications [2], [4].

Vue essentially strikes a balance between ease of use and advanced functionality. This makes it very versatile. The combination of two-way data binding, component-based architecture and support tools make Vue a strong contender for a frontend framework, despite it not being adopted as widely as Angular or React [2], [4].

Backend Technologies

1. Flask

Flask is a lightweight microframework for Python. It was designed to allow developers to build web applications quickly. Flask is built around the WSGI toolkit and the Jinja2 web template engine. This means that it focuses on providing only the essential components for web development. The minimalistic approach gives developers complete control over the application's structure. Additional tools and libraries can be integrated as needed [6], [7].

Being a microframework, Flask provides core functionality, like routing, request handling, and templating. It leaves out complex features, like database support, to be handled by optional extensions. This "bare bones" philosophy makes Flask a good choice for developers who want more control over their applications, deciding how most things are done by themselves, rather than relying on the framework to handle it. The minimalist approach also allows for quick setup and development with minimal boilerplate code [7].

Some more of Flask's features include RESTFUL request dispatching, secure cookies for session management, and template inheritance. These features make it well suited for small applications. It is also suitable for cases where customization is highly valued [6], [7].

Flask's flexibility comes with trade-offs. It lacks built in support things like databases and user authentication, crucial features for large scale applications. To achieve these features, developers must rely on third-party libraries or create their own versions. While very flexible, Flask's disadvantage here is that it can increase the amount of time and effort required for creating and maintaining certain functionalities that are considered essential for medium to large scale applications [7].

Being so popular, Flask is very well documented and has a very active community. This makes Flask accessible to developers with varying skill levels. Beginners will appreciate the easy setup while more experienced developers will value Flask's flexibility and control. The easy setup in particular makes it a preferred choice for educational purposes [6].

In summary, Flask is a Python framework for developers who want simplicity, flexibility, can control over finer details in their application. The minimalist design makes it ideal for small applications, but the need for third-party components for advanced features may require additional effort for integrating and manging said features, hurting Flask as a contender for larger applications [6], [7].

2. Django

Django is also a Python based backend framework. Unlike Flask, it was designed to simplify the development of applications by providing a suite of built in tools. This approach is called a “batteries-included” framework. By having native support for features like user authentication and databases, Django developers have no need for third-party tools. Some other built-in features include Object Relational Mapping (ORM), form handling, and an admin interface. The admin interface in particular is a very powerful tool as it can be used manage the content of a connected database and finding out what APIs are exposed, how to call them, and with what parameters [6].

Django follows a Model-View-Template (MVT) architecture. This allows for a separation of concerns between the three components. The ORM allows for interaction with databases. This is a very useful feature as it allows developers to define models in Python and Django automatically generates the corresponding database tables. This significantly simplifies database development and management [6].

Django is also very scalable and secure, making it a preferred choice for large scale applications. Some security features include Cross Site Request Forgery, Cross Site Scripting, and SQL injection. Django also provides tools for session management and role-based access, allowing, for example, application staff to access parts of the application top accessible to common users [6].

Django’s templating engine makes creating dynamic web pages relatively simple. It combines the backend data with reusable HTML templates, allowing developers to keep a consistent design across the application. The use of templates also reduces redundancy [6].

Despite Django’s numerous advantages, its monolithic design and opinionated approach can be a challenge for developers who value simplicity and flexibility in the backend framework. Django follows the Don’t Repeat Yourself (DRY) principle and prioritizes convention over configuration. This may result in a steeper learning curve than Flask. Also, Django’s toolkit, while highly valued for complex applications, can feel slightly overengineered and unnecessary for smaller applications [6].

Like Flask, Django also has extensive documentation and a large, active community. This provides developers with support and additional plugins should they need them. Django is well suited for applications where scalability and fast development are important. These can include social media platforms and e-commerce websites [6].

In summary, Django is feature rich backend framework that is perfect for building complex, high performance applications quickly. Being “batteries included” makes sure that developers have access to all the basic features required for a complex app. However, for small or flexible applications, the extensive toolkit can feel more like a disadvantage rather than an advantage [6].

Databases

1. MongoDB

MongoDB is a NoSQL, document-based database. Like the term implies, MongoDB does not use any SQL. Instead, it organizes data into a Binary JSON (BSON) format. This enables MongoDB to have a schema-less structure. This allows developers to handle unstructured and semi-structured data without the need for a schema. The flexibility makes MongoDB well suited for modern applications [8], [9].

MongoDB achieves horizontal scalability through sharding. Sharding is when data is spread out over serve servers or clusters, allowing MongoDB to handle large datasets. The scalability that this offers allows applications to manage large workloads by simply adding more nodes to a cluster. This makes MongoDB ideal for applications that need extensive data storage [8], [9].

MongoDB also supports indexing on embedded documents and geospatial queries. These additional features increase performance for specific workloads. For example, geospatial querying makes MongoDB well suited in apps like, for example, logistics applications. MongoDB's features make it efficient at operations like data insertion and real time querying [8].

MongoDB also includes advanced features such as indexing on embedded documents, geospatial queries, and aggregation pipelines, which enhance its performance for specific workloads. These capabilities make MongoDB highly efficient for operations such as data insertions, real-time updates, and basic querying. For instance, geospatial queries enable applications to perform location-based searches, a critical feature in sectors like logistics [8].

The NoSQL approach might seem appealing but has some trade-offs when compared to traditional relational databases. Without a structured relationship, MongoDB struggles with complex queries like multi-table joins. These are supported natively by relational databases like PostgreSQL [9].

MongoDB is also compatible with many programming languages and platforms, which makes it accessible to most developers. No matter what programming language they choose, there is a high chance that they will be able to use MongoDB [9].

MongoDB Atlas is a cloud-based solution for managing a MongoDB database. It simplifies deployment and scaling, but without careful planning, can also lead to inefficient queries or increased storage cost [9].

With MongoDB's schema-less design and excellent horizontal scalability, it is a strong choice for applications that need flexibility and performance. It is very good at handling large and distributed datasets and modern workloads. While it does excel in these cases, in does not have much to offer over other contenders in conventional scenarios [8], [9].

2. PostgreSQL

PostgreSQL is an open source relational database. It is very robust and scalable. It was designed to handle complex data structures and operations. PostgreSQL follows the Atomicity, Consistency, Isolation, and Durability (ACID) principles. These make sure that the data is reliable and that data integrity is kept. This makes PostgreSQL a preferred choice for enterprise developed apps for various industries [9].

PostgreSQL can handle both structured and semi-structured data quite well. It bridges the gap between relational databases and NoSQL ones by having support for JSON and JSONB datatypes. The versatility is enhanced by extensions like PostGiS which adds geospatial capabilities similar to MongoDB. PostgreSQL can also optimize query performance for different workloads by using B-Tree, and GIN and BRIN indexes. For example, GIN is very effective for text search. B-Tree on the other hand is very good with numbers, making suitable for transaction systems [9].

PostgreSQL outperforms many NoSQL databases in multi-table joins, nested queries, and other complex queries. It has support for foreign keys, which makes it very useful for applications where high normalization and inter-table relationships are present. These strengths make PostgreSQL suitable for enterprise systems and data warehousing [9].

PostgreSQL relies on vertical scaling. This means that its performance improves by upgrading server hardware. This can become a bottleneck in some scenarios, when compared to the horizontal scalability of NoSQL databases like MongoDB [9].

Developers can also add custom functions and datatypes, making queries more efficient [9].

The various features of PostgreSQL make it very popular, one of the most popular databases in the world. It has regular updates, extensive documentation and an active community. Its open source nature also allows users to customize their databases however they need [9].

PostgreSQL is best suited for handling complex queries. While its vertical scaling is a limiting factor to its performance, its many advanced features make it a solid choice for data driven applications [9].

3. SQLite

SQLite is a lightweight, embedded database engine. It's known for its simplicity and portability. Unlike traditional databases that are server based, e.g. PostgreSQL, SQLite is file-based and runs inside the host application. This self-contained approach makes SQLite very portable. It also makes it easy to deploy and manage, as there is minimal configuration and maintenance. These make SQLite very popular for mobile applications and small-scale projects. SQLite is also ACID complaint [10].

The file-based approach allows developers to store and distribute their database as a single file. This makes it perfect for applications where a server infrastructure is impractical, like in standalone desktop applications [10].

SQLite features fast read-write performance for single-user scenarios. This also makes it a good choice for mobile applications [10].

Though, SQLite does have some disadvantages. It has limitations in high concurrency or large-scale applications. It also lacks features like parallel query execution and complex indexing. This makes it less suitable for applications that expect to have the database being accessed by multiple users simultaneously. It also does not support features like clustering or sharding making it even less suitable for large-scale applications [10].

SQLite's limitations are counterbalanced by is simplicity and resource efficiency. It also has a small footprint, typically less than 1MB. This makes it ideal for embedded systems and applications where hardware resources are expected to be limited [10].

Its lack of server also reduces operational complexity and cost. This makes it a suitable choice for developers who want a lightweight data storage solution [10].

Supported by high-quality documentation, cross-platform compatibility, and large community, SQLite is used across a large range of industries and applications. While it's not designed for high performance, multi-user systems, its versatility and ease of integration have led it to become the most deployed database system in the world [10].

2.4. Other Research

OpenAI GPT Models

OpenAI's Generative Pre-trained Transformer (GPT) models represent a significant advancement in natural language processing and artificial intelligence. Especially the latest GPT-4 model. These models, as the name implies, are pre-trained on large publicly available datasets. They utilize unsupervised learning during pre-training and are late fine tuned with techniques like Reinforcement Learning form Human Feedback (RLHF) [11].

GPT-4 is a multimodal model. This means that it can process text and images. It shows human-level performance in a large number of professional and academic benchmarks. For example, it scored in the top percentile in the Uniform Bar Examination, which is an exam that is designed to test the knowledge and skills of a lawyer before they become licensed to practice law in the United States [11].

GPT-4's pre-training makes it powerful in applications like dialogue systems and text summarization. It also significantly improved its factual accuracy compared to its predecessor, GPT-3.5 [11].

One of GPT-4's features is its ability to interpret and generate text based on visual inputs, as shown in tasks like creating working code from a hand-drawn sketch. This increases its utility in areas like software development and education. For instance, organizations like Khan Academy use GPT-4 to create Khanmigo, an AI tutor [12] [17].

Despite its advancements, GPT-4 faces challenges such as "hallucinations" , where the model generates inaccurate or nonsensical information. It doesn't learn from experience and relies entirely on its pre-trained knowledge base, which cuts off in September 2021 [11], [12].

OpenAI has implemented strict guardrails to address ethical concerns, like as refusing to provide outputs for sensitive or harmful prompts. However, ongoing issues like model biases and vulnerabilities to exploits highlight the need for further research and transparency in AI development [11], [12].

Natural Language Processing (NLP)

Natural Language Processing (NLP) is a subset of artificial intelligence and linguistics. NLP specialises in allowing computers to understand, interpret, and generate human language. NLP is becoming more important and prevalent with the large amount of data stored, such as books, reports, articles, and other forms of online content, which computers to retrieve meaningful information back to the user [13].

NLP can be traced back to the 1950s when Alan Turing came up with the “Turing Test”, a way to measure the ability of a machine to behave in an intelligent way, similar to humans [13].

NLP has two main parts: natural language understanding (NLU) and natural language generation (NLG). NLU is the analysis stage where the input text is analysed, and intent and meaning are derived from it. NLG is the stage where the system used the information from the analysis of the input text to create an appropriate response to the user [13].

NLP has several key phases

- Morphological and Lexical Analysis:
  Identifies the structure of words and splits text up into meaningful units [13].
- Syntactic Analysis:
  Examines grammatical structures to understand relationships between words [13].
- Semantic Analysis:
  Derives contextually accurate meanings of words and sentences [13].
- Discourse Integration:
  Links sentences to understand their collective meaning [13].
- Pragmatic Analysis:
  Interprets the intent and implied meanings based on real-world knowledge [13].

NLP applications include machine translation and question-answering systems. Modern NLP relies on machine learning to develop adaptable systems that learn patterns from large datasets. [13].

GDPR

The General Data Protection Regulation (GDPR) is a privacy law that applies to any organization if that organization processes data from people in the European Union. GDPR was put into effect in May of 2018. With the creation of GDPR, Europe is "signalling its firm stance on data privacy and security at a time when more people are entrusting their personal data with cloud services and breaches are a daily occurrence" [14].

Data Processing is defined by the GDPR as any action performed on data. Common examples are collecting, storing, and analysing [14].

GDPR involves seven key data protection principles:

1. Lawfulness, fairness, and transparency
   Processing of the data cannot be used of illegal activities, and purpose of processing must be made clear to the person whose data is to be processed [14].
2. Purpose Limitation
   The data cannot be used for anything that the data subject was not made aware of [14].
3. Data Minimization
   The amount of data processed should be as small as possible to accomplish the desired purpose [14].
4. Accuracy
   Personal data must be kept accurate and up to date [14].
5. Storage Limitation
   Personal data can only be stored for as long as necessary for the purpose. It cannot be stored once it has served its purpose and must be deleted [14].
6. Integrity and Confidentiality
   The processing of data must be secure [14].
7. Accountability
   The responsibility of compliance to the GDPR principles is solely that of the data controller [14].

With GDPR, every new system that processes data must incorporate the date protection principles "by design and by default" [14].

GDPR also covers when the processing of data is allowed. These include:

- When the data subject has consented to having their data processed [14].
- When data processing is required for a contract where one of the parties is the data subject [14].
- When data processing is required to comply with legal obligations [14]
- When the data subject's life is in danger and processing their data will save their life [14].
- When data processing is used to "perform a task in public interest" [14].
- When the data controller has a legitimate, lawful interest in the data subject's data. This is instance is listed as flexible but can be nullified by the "fundamental rights and freedoms of the data subject" [14].

Chatbots

A conversational agent, commonly known as a chatbot, is a piece of software that simulates a human conversation either through text or voice. They have become essential in applications like customer service because of their ability to handle queries efficiently [15], [16].

Chatbots use natural language processing and machine learning, which allow them to understand user intent and generate an appropriate response. NLU and NLG are important parts needed for chatbot to function in a conversation. Platforms like Botpress are commonly used for building and deploying chatbots due to their integration-friendly design and built-in NLP modules [15].

Recent trends in chatbot development include personalization and multimodal interactions. Chatbot systems designed for education have shown accuracy rates of over 90%, proving their effectiveness in providing students with high quality information. These chatbots utilize structured knowledge bases and advanced intent recognition techniques to enhance user engagement [15].

Despite these advancements, chatbots face challenges such as handling ambiguous or contextually complex queries. Ensuring scalability and maintaining high accuracy across languages and domains remain areas for ongoing research. Ethical considerations, like data security for example, are also important as chatbots handle more sensitive information with each new development [15], [16].

The future of chatbots lies in integrating advanced AI features such as voice recognition, multimodal interactions, and deep learning techniques. These advancements show promise in enhancing their capability to produce human conversations [16].

2.5. Existing Final Year Projects

Project 1

Title: Student Helper - A Chatbot Assistant for Students

Student: Padraig McCarthy

Description: This project involves the development of a chatbot designed to assist students at TU Dublin by providing information about their college, such as timetables and FAQs. The chatbot interacts with users via natural language, making it easier to access college-related data through familiar platforms like Facebook Messenger. In addition to answering queries, the chatbot includes a reminder feature that notifies students when it's time to leave for their classes, enhancing the user experience by offering proactive assistance.

What is complex in this project: The complexity in this project arises primarily from integrating natural language processing (NLP) with a chatbot that interfaces with students for retrieving timetable and FAQ data. The project had to develop a robust system for data gathering, particularly scraping data from public websites and cleaning it for use. Understanding users' messages accurately and processing natural language inputs also posed challenges. Implementing accurate and helpful reminder services based on students' timetables further added to the complexity.

What technical architecture was used: The project employed a combination of technologies, including a Java-based backend utilizing the SpringBoot framework for API control and database transactions. MySQL was chosen for relational data, while ElasticSearch was used for efficient data retrieval and search operations. DialogFlow was utilized as the NLP service to process user inputs, and Facebook Messenger was chosen as the messaging platform for chatbot interaction.

Key strengths and weaknesses of this project: A key strength of the project is its use of well-established technologies like ElasticSearch, SpringBoot, and DialogFlow, which enhanced data handling, scalability, and natural language understanding. The project's architecture also allowed for timely retrieval of timetable data, making it user-friendly for students. However, one notable weakness was the difficulty in gaining access to institutional data, leading to the need for time-consuming data scraping. Additionally, while the project functioned well for proof-of-concept, scaling it beyond a single course could require further development.

Project 2

Title: Student Organizer

Student: Behzad Sanehi

Description: This project, titled "Student Organizer," is a mobile application aimed at simplifying student life by helping them manage their class timetables, events, and tasks. The app integrates functionalities such as timetable views, event creation, event sharing, group chats, and notes, all in one place. It allows students to organize their academic and social lives more efficiently by centralizing these features in a user-friendly interface.

What is complex in this project: The complexity in this project mainly stems from integrating multiple features such as event sharing, group messaging, and calendar management within a single mobile app. Additionally, using Firebase Cloud Messaging to handle event sharing, even when users are offline, added significant technical complexity. Ensuring that shared events reach all intended users asynchronously, regardless of their online status, introduced challenges that required careful management of real-time data synchronization between devices and servers.

What technical architecture was used: The project used a three-tier architecture, with an Android client as the front-end, a Node.js server running on DigitalOcean as the backend, and MongoDB for data storage. The backend also employed Firebase Cloud Messaging (FCM) for handling notifications and event sharing. SQLite was used locally on the Android device for storing timetable data to enable offline viewing. The use of Retrofit for network communication with the server enabled seamless data retrieval and posting between the app and the server.

Key strengths and weaknesses of this project:

One of the main strengths of this project is its architecture, which enabled efficient real-time event sharing through Firebase Cloud Messaging. This ensured that events were delivered to users even when their devices were offline. The use of well-established technologies like Node.js, Express.js, and MongoDB contributed to the app's scalability and efficient data handling. Additionally, the integration of Retrofit and Firebase improved user interaction, making data retrieval and communication between the app and server smooth. The inclusion of offline functionality for viewing timetables also increased usability, allowing students to access their schedules without an internet connection.

However, the project also had some weaknesses. A key challenge was that the student had to work with unfamiliar technologies like MongoDB and Node.js, which required time to learn and implement effectively. This steep learning curve added complexity to the development process. Furthermore, synchronizing data between the local SQLite database and the online database posed technical challenges, particularly when users modified events. Another drawback was that the development of unplanned features, while beneficial, led to delays that could have been avoided with more careful project planning.

2.6. Conclusions

The literature review covered various potential technologies that could be used in developing the application, as well as other domain-specific research relevant to the project. It concluded with a justification for the final selection of technologies.

For the backend, Django was selected. Django provided a wide range of built-in features such as native database support and user authentication. These features proved to be particularly useful during the development of the prototype. While some of Django's more advanced features were not utilized, the inclusion of essential functionality out of the box saved valuable development time that was instead focused on implementing the core feature of the project – the chatbot assistant.

For the frontend, React was chosen. React's popularity, extensive documentation, and community support made it an ideal choice for the project. It was well-suited for building a lightweight, single-page application, and its performance benefits due to the virtual DOM contributed to a smoother user experience.

Regarding the database, a split approach was taken. While PostgreSQL was selected for the final version of the application due to its scalability and support for concurrent users, SQLite was used during early development and prototyping. Since the prototype only needed to demonstrate core features in a single-user context, SQLite offered a lightweight and efficient solution.

The chatbot was developed using the Botpress framework. Botpress's graphical interface enabled intuitive chatbot design and integration with the application's APIs. Its built-in emulation mode also supported iterative testing and refinement of conversational flows.

Finally, the application was designed with data security in mind. For instance, when a user deleted their account, all associated data – including tasks and events – was removed from the database. Additionally, user passwords were securely stored using hash values to ensure proper protection of user credentials.

3. Software Design

3.1 Introduction

This chapter outlines the architectural and technical design decisions made during the development of the calendar and task management application with an integrated AI chatbot assistant. It describes the selected software methodology, provides an overview of the system's components, and highlights the key design choices that enabled seamless interaction between the user, the interface, and the chatbot. The design phase was crucial in ensuring that the application would be scalable, secure, and user-friendly while supporting both manual and conversational interaction modes.

3.2. Software Methodology

The project followed an agile-inspired, incremental development approach. Agile was chosen due to its flexibility and adaptability, which were particularly valuable given the experimental nature of integrating a chatbot assistant with natural language capabilities. Unlike traditional waterfall methodologies, which require fixed planning upfront, agile enabled iterative progress, allowing for the gradual evolution of the system through short development cycles and continuous feedback.

This approach was especially appropriate for a Final Year Project, where requirements could shift slightly based on technical feasibility, supervisor feedback, or user testing. The nature of chatbot development — involving frequent fine-tuning of responses, logic, and API interactions — also made agile a practical choice, as it allowed these refinements to be implemented incrementally without needing to rework the entire system.

Although the project did not follow a formal agile process with full sprint ceremonies, several agile principles were applied:

- Development was broken down into distinct phases, each with specific short-term goals (e.g., account creation, event CRUD, chatbot integration, testing).
- Tasks were tracked informally using a to-do list and development log, which helped prioritize features and bugs.
- Regular self-review sessions were held at the end of each development milestone to evaluate progress and adjust plans accordingly.
- Prototypes were tested frequently, especially the chatbot module, with feedback guiding improvements.
- Changes were deployed and tested incrementally, allowing features to be validated in isolation before being fully integrated into the main system.

The initial phase focused on implementing core functionality such as user authentication and CRUD operations for tasks and events. These features formed the foundation of the application. Once stable, the focus shifted toward chatbot integration. This involved both the design of conversation flows in Botpress and the backend development of API endpoints that the chatbot could call. As each chatbot intent or interaction was developed, it was tested independently before being linked with the main application interface.

By using an agile-inspired approach, the project maintained development momentum, responded quickly to unforeseen challenges (e.g., chatbot misunderstandings or frontend/backend communication issues), and allowed for a gradual, test-driven refinement of key features.

3.3. Requirements Gathering

Requirements were identified through a combination of research into existing task management tools and the analysis of user expectations from intelligent assistant systems. The goal was to build an intuitive and lightweight productivity tool that allowed users to manage their schedules either manually or through natural language input.

The key functional requirements were:

- User Authentication: Secure login/logout and access control.
- Manual Task/Event Management: CRUD operations for tasks and calendar events.
- AI Chatbot Interaction: Support for natural language commands to perform CRUD actions.
- Event Summarization and Suggestions: The chatbot should summarize events and suggest alternative scheduling when needed.
- Data Privacy and Security: User data should be protected, with passwords hashed and all user data deleted upon account deletion.

Non-functional requirements included:

- Responsiveness: The application should not have long loading or wait times.
- Usability: The UI should be clean and intuitive.
- Modularity: Backend and frontend should be loosely coupled for easier updates and testing.

3.4. Overview of System

The frontend was built using React with Vite, enabling the creation of a modern and responsive user interface. Thanks to React's component-based architecture, code reusability was achieved, which contributed to faster development and easier maintenance.

The backend was developed using Django, which handled the core system logic such as user authentication and database operations. A set of RESTful APIs was implemented and exposed to allow users to perform CRUD operations on tasks and calendar events. These APIs were accessed from the frontend, either through manual user interaction or via the chatbot.

For data storage, the application uses PostgreSQL. PostgreSQL was chosen due to its robustness, scalability, and suitability for applications supporting multiple concurrent users. Its support for advanced queries and data integrity features made it a good choice for the project's requirements.

The chatbot was developed using the Botpress framework. Botpress provided built-in natural language processing (NLP) capabilities, which made it possible to design a conversation-capable assistant. Additionally, Botpress allowed for seamless integration with the application's backend APIs, enabling the chatbot to perform actions such as creating or modifying calendar events based on user input.

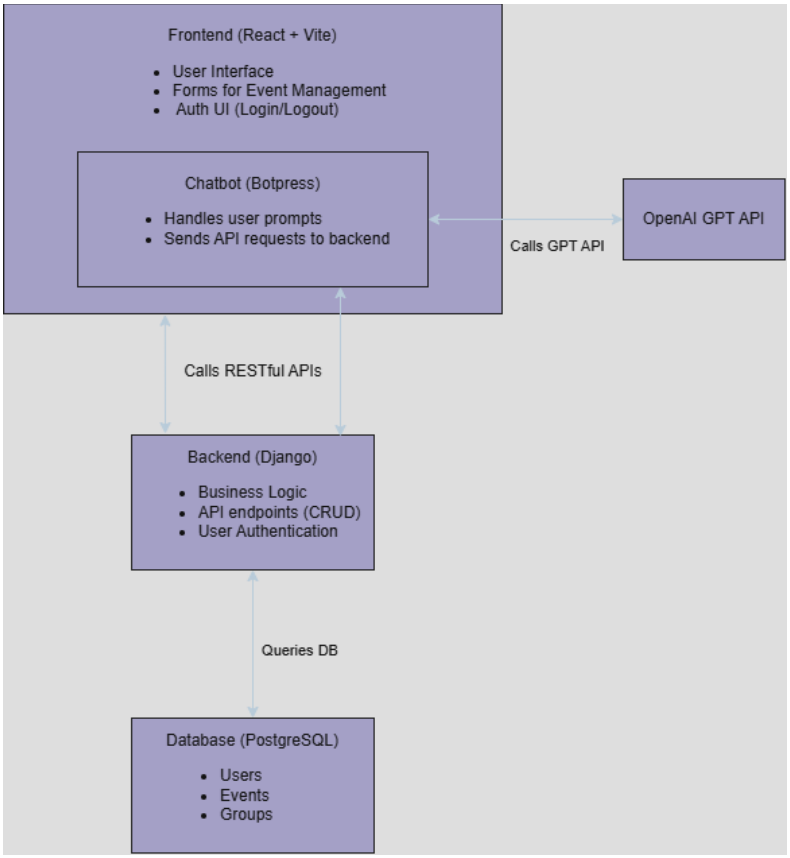


Figure 3.1 - illustrates the core architecture of the application, including the React frontend, Django backend, PostgreSQL database, and the integration of a chatbot powered by Botpress and an OpenAI GPT model.

3.5. Use Cases

The primary actor in the system is the User, who interacts with two key functional areas: Events and Groups. Users can perform standard operations such as creating, viewing, updating, and deleting both events and groups. These operations are supported by a set of RESTful API endpoints exposed by the backend and consumed by the frontend or the chatbot assistant.

Two types of interaction paths are supported for each feature set:

- Manual interaction through the graphical user interface
- Conversational interaction via the chatbot assistant

In both cases, the application ensures the correct API calls are made to carry out the requested actions, regardless of how the command was issued. The chatbot layer, implemented using Botpress and GPT, acts as an intermediary that interprets user commands and triggers the appropriate backend logic.

Figure 3.2 illustrates the use case diagram for the system. The diagram highlights the following use cases:

- Interact with Events and Interact with Groups represent the main user activities.
- Each of these includes calling the relevant backend APIs (<<includes>>) to perform the associated CRUD operations.
- The ability to Type to Chatbot Assistant is modelled as an extension (<<extends>>) of each interaction, indicating that natural language input is an optional enhancement to the manual interface.

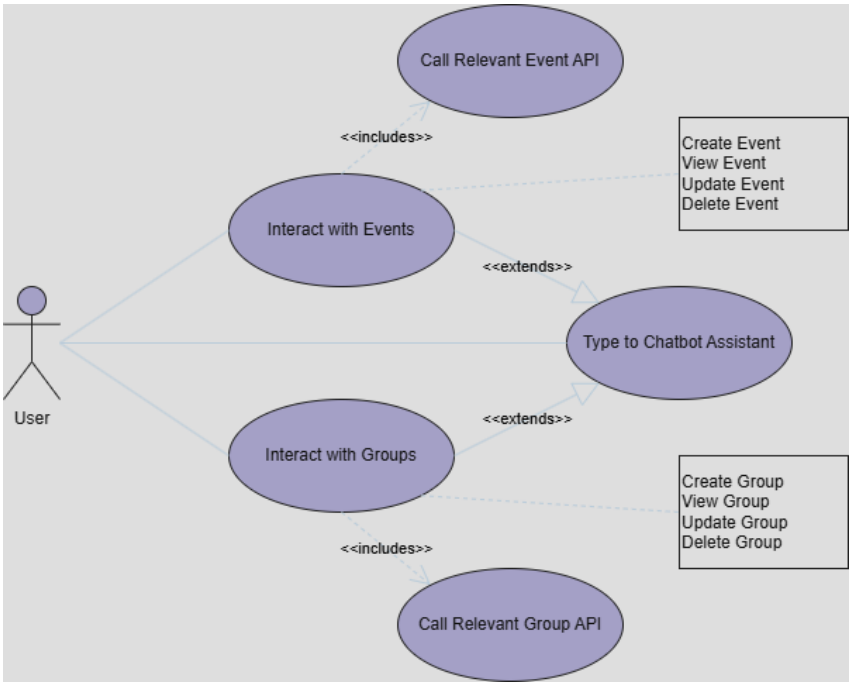


Figure 3.2 - Use Case Diagram for User Interactions with Events and Groups

3.6. Database Overview

Figure 3.3 shows the entity relationship diagram (ERD) for the PostgreSQL database used in the application. The schema includes three main tables: users_customuser, mycalendar_taskeventgroup, and mycalendar_tasksevents.

The users_customuser table stores registered user accounts and includes fields such as username, email, and password. For security, passwords are stored as hashed values using Django's built-in authentication system. This table also includes standard metadata such as is_staff, is_active, and date_joined.

Each task or event created by a user is stored in the mycalendar_tasksevents table. This table includes details such as the task/event name, start and end datetime, description, and timestamps for when it was created and last modified. Each task/event is linked to its owner via the owner_id field, enforcing user-specific data access. Tasks/events can also be optionally grouped using the group_id foreign key, which links to the mycalendar_taskeventgroup table.

The mycalendar_taskeventgroup table allows users to organize tasks and events into color-coded groups. Each group includes a name, description, color, and an owner_id field to associate it with a specific user.

This schema ensures that each user has access only to their own tasks and groups, supporting personalized calendar management within the application.

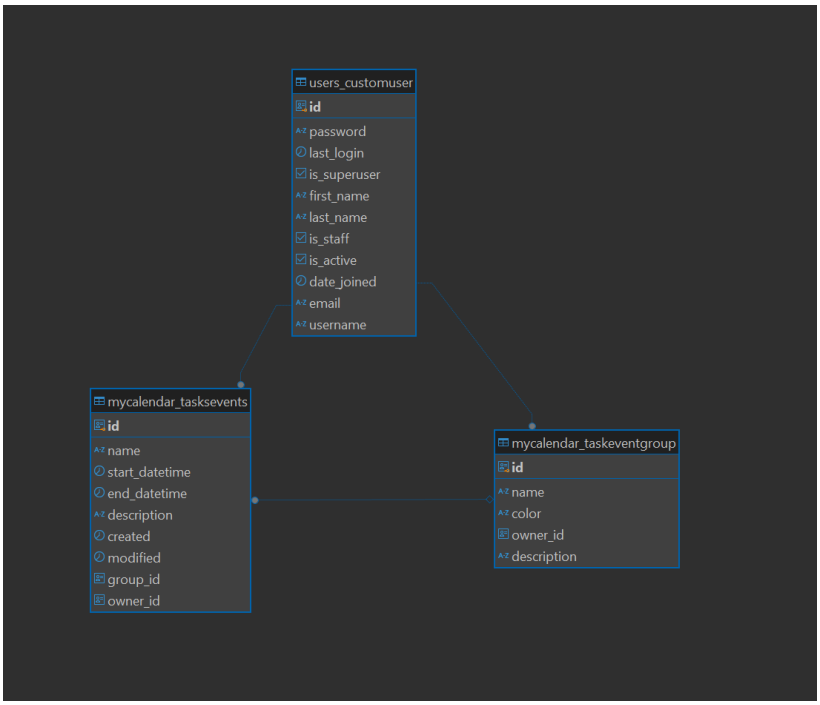


Figure 3.3 - represents the relational structure of the PostgreSQL database used in the application. It shows how users, event groups, and individual events are linked through foreign key relationships.

3.7. Feature Prioritization

User Authentication	Must Have	Register, login, logout, etc.
CRUD operations	Must Have	Add, edit, view, and delete tasks/events
Basic Chatbot Integration	Must Have	Handle CRUD operations via chatbot
Advanced Chatbot Commands	Should Have	Multi-step commands, like rescheduling events
Grouping	Could Have	User categorization of tasks/events
Sorting	Could Have	Sort tasks/events by date range, status, priority, group, etc.
Reminders	Could Have	Notifications for upcoming events and deadlines (via email)
Multi Language Support	Won't Have	Application will only be in English
Advanced AI Features	Won't Have	Chatbot will not have complex features like sentiment analysis.
Enterprise-Level Features	Won't Have	No multi-user tasks, collaboration, etc.

Figure 3.4

Figure 3.4 shows a MoSCoW prioritization table for the application's features. The application will have user authentication, task/event creating, viewing, editing, and delete, both manually and via the chatbot as core features. More complex chatbot operations are high priority features.

Development of features like grouping, sorting, and reminders can begin once the higher priority features are implemented. These are "nice to have" features and are common in existing calendars and task managers but are not the main focus of the project.

Features like multi-language support, advanced ai, and enterprise-level features are beyond the scope of this project and will not be implemented into the application.

3.8. Conclusions

The design phase of the application laid a foundation for the implementation of a task and calendar management system. By using an agile-inspired methodology, development was approached incrementally, which allowed for flexibility in incorporating feedback and adapting to the evolving technical requirements of integrating a chatbot assistant.

Functional and non-functional requirements were identified to ensure the system met user needs while remaining scalable, modular, and secure. The chosen technologies—React with Vite for the frontend, Django for the backend, PostgreSQL for data storage, and Botpress with GPT for natural language processing—were effective and complementary components in implementing the intended functionality.

The architectural overview, use case analysis, and database design showed a well-integrated system that allowed users to interact with their calendar either manually or through conversational input. The chatbot was added as an enhancement to the user experience rather than a replacement for traditional input methods.

4. Software Development

4.1. Introduction

This chapter outlines the implementation process of the calendar and task management application, highlighting the key technologies, tools, and coding practices used throughout development. It details the creation of the frontend and backend, the integration of the chatbot assistant, and the implementation of supporting features such as user authentication, API endpoints, and data handling. The development phase followed the design specifications outlined in Chapter 3, with a focus on delivering a functional and user-friendly system through iterative development.

4.2. Software Development

The application was developed as a full-stack web platform, with the backend, frontend, and chatbot layers implemented as separate components. Development began with setting up the core project structure, followed by incremental implementation of features.

Backend Development (Django)

The backend was built using the Django web framework, which provided built-in features for user authentication, database management, and REST API development through Django REST Framework (DRF). Key development milestones included:

- **User Authentication:**
  Django's authentication system was extended with a custom user model (users_customuser) to create custom login using the email and password. Secure password handling was ensured using Django's built-in hashing mechanisms. Token-based authentication was implemented using Knox for logging in.

```python
class CustomUserManager(BaseUserManager):
    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError('Email is a required field')

        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        user.set_password(password) # change password to hash
        user.save(using=self._db)
        return user

    def create_superuser(self, email, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)
        return self.create_user(email, password, **extra_fields)

class CustomUser(AbstractUser):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    email = models.EmailField(unique=True)
    username = models.CharField(max_length=150)
    objects = CustomUserManager()

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []
```

Figure 4.1 – custom user model

```python
class LoginViewset(viewsets.ViewSet):
    permission_classes = [permissions.AllowAny]
    serializer_class = LoginSerializer

    def create(self, request):
        serializer = self.serializer_class(data=request.data)
        if serializer.is_valid():
            email = serializer.validated_data['email']
            password = serializer.validated_data['password']

            user = authenticate(request, email=email, password=password)
            if user:
                _, token = AuthToken.objects.create(user)
                return Response(
                    {
                        "user": self.serializer_class(user).data,
                        "token": token
                    }
                )
            else:
                return Response({"error": "Invalid credentials"}, status=401)
        else:
            return Response(serializer.errors, status=400)
```

Figure 4.2 – Login Viewset creates a session for the user when correct credentials are provided

- **CRUD Operations for Events and Groups:**
  Models and RESTful endpoints were developed for managing calendar events and groups. The backend was built using Django and Django REST Framework, allowing for the creation of modular and secure API endpoints that support full CRUD (Create, Read, Update, Delete) operations. Ownership validation was a core component throughout the API logic to ensure that users could only access and manipulate their own data.

  Each API method handles data input validation, user ownership checking, and structured JSON responses:

  - **Create Operations:**
    When creating an event or group, the system first checks if the request includes a valid owner_id. If the owner exists, the serializer validates the input and saves the object, assigning the user as the owner. Errors such as missing owners or invalid input return appropriate 400 or 404 responses.
  - **List Operations:**
    Users can retrieve a list of their own events or groups by passing their owner_id as a query parameter. If no owner is provided, all records are returned (primarily for administrative or debugging purposes). Ownership filtering ensures that users only see their own data.
  - **Retrieve Operations:**
    The API retrieves a specific event or group by its primary key and verifies that it belongs to the requesting user. If the object does not exist or the user is not authorized to view it, the API responds with 404 Not Found or 401 Unauthorized respectively.
  - **Update Operations:**
    When updating a task or group, the system again verifies ownership before applying changes. Data is partially updated using the PUT method, and the updated object is serialized and returned upon success.
  - **Delete Operations:**
    Similarly, before deletion, ownership is validated to prevent unauthorized access. Upon successful deletion, a 204 No Content or a success message is returned.

  Additional logic was included in the group-related endpoints to enhance functionality:

  - Each group response includes a list of event IDs associated with it, making the frontend's job of organizing and displaying grouped events easier.
  - Group-based responses are post-processed before being returned, enhancing the output with event linkage data.

```python
class TasksEvents(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    owner = models.ForeignKey('users.CustomUser', on_delete=models.CASCADE, related_name='tasks_events')
    name = models.CharField(max_length=200)
    start_datetime = models.DateTimeField(default=now)
    end_datetime = models.DateTimeField(blank=True, null=True)
    description = models.TextField(blank=True, null=True)
    group = models.ForeignKey('TaskEventGroup', on_delete=models.SET_NULL, related_name='events', blank=True, null=True)
    created = models.DateTimeField(auto_now_add=True)
    modified = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.name

class TaskEventGroup(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    owner = models.ForeignKey('users.CustomUser', on_delete=models.CASCADE, related_name='task_event_groups')
    name = models.CharField(max_length=200)
    description = models.TextField(blank=True, null=True)
    color = models.CharField(max_length=100)

    def __str__(self):
        return self.name
```

Figure 4.3 – event and group models

```python
def create(self, request):
    owner_id = request.data.get('owner')
    if not owner_id:
        return Response({'error': 'Owner is required'}, status=400)
    try:
        owner = CustomUser.objects.get(id=owner_id)
    except CustomUser.DoesNotExist:
        return Response({'error': 'Owner not found'}, status=404)
    serializer = self.serializer_class(data=request.data)
    if serializer.is_valid():
        serializer.save(owner=owner)
        return Response(serializer.data, status=201)
    return Response(serializer.errors, status=400)
```

Figure 4.4 – create event endpoint

```python
def list(self, request):
    owner_id = request.query_params.get('owner')
    if not owner_id:
        queryset = TasksEvents.objects.all()
    else:
        try:
            owner = CustomUser.objects.get(id=owner_id)
        except CustomUser.DoesNotExist:
            return Response({'error': 'Owner not found'}, status=404)
        queryset = TasksEvents.objects.filter(owner=owner)
    serializer = self.serializer_class(queryset, many=True)
    return Response(serializer.data)
```

Figure 4.5 – view all events endpoint

```python
def retrieve(self, request, pk=None):
    owner_id = request.query_params.get('owner')
    if not owner_id:
        return Response({'error': 'Owner is required'}, status=400)
    try:
        owner = CustomUser.objects.get(id=owner_id)
    except CustomUser.DoesNotExist:
        return Response({'error': 'Owner not found'}, status=404)
    try:
        task_event = TasksEvents.objects.get(pk=pk)
    except TasksEvents.DoesNotExist:
        return Response({'error': 'TaskEvent not found'}, status=404)
    if task_event.owner != owner:
        return Response({'error': 'Unauthorized'}, status=401)
    serializer = self.serializer_class(task_event)
    return Response(serializer.data)
```

Figure 4.6 – view one event endpoint

```python
def destroy(self, request, pk=None):
    owner_id = request.query_params.get('owner')
    if not owner_id:
        return Response({'error': 'Owner is required'}, status=400)
    try:
        owner = CustomUser.objects.get(id=owner_id)
    except CustomUser.DoesNotExist:
        return Response({'error': 'Owner not found'}, status=404)
    try:
        task_event = TasksEvents.objects.get(pk=pk)
    except TasksEvents.DoesNotExist:
        return Response({'error': 'TaskEvent not found'}, status=404)
    if task_event.owner != owner:
        return Response({'error': 'Unauthorized'}, status=401)
    task_event.delete()
    return Response({'message': 'TaskEvent deleted successfully'}, status=204)
```

Figure 4.7 – delete event endpoint

```python
def update(self, request, pk=None):
    owner_id = request.query_params.get('owner')
    if not owner_id:
        return Response({'error': 'Owner is required'}, status=400)
    try:
        owner = CustomUser.objects.get(id=owner_id)
    except CustomUser.DoesNotExist:
        return Response({'error': 'Owner not found'}, status=404)
    try:
        task_event = self.queryset.get(pk=pk)
    except TasksEvents.DoesNotExist:
        return Response({'error': 'TaskEvent not found'}, status=404)
    if task_event.owner != owner:
        return Response({'error': 'Unauthorized'}, status=401)
    serializer = self.serializer_class(task_event, data=request.data, partial=True)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=200)
```

Figure 4.8 – update event endpoint

```python
def create(self, request):
    owner_id = request.data.get('owner')

    if not owner_id:
        return Response({'error': 'Owner is required'}, status=400)

    try:
        owner = CustomUser.objects.get(id=owner_id)
    except CustomUser.DoesNotExist:
        return Response({'error': 'Owner not found'}, status=404)

    serializer = self.serializer_class(data=request.data)
    if serializer.is_valid():
        serializer.save(owner=owner)
        return Response(serializer.data, status=201)
    return Response(serializer.errors, status=400)
```

Figure 4.9 – create event group endpoint

```python
def list(self, request):
    owner_id = request.query_params.get('owner')
    if not owner_id:
        queryset = TaskEventGroup.objects.all()
    else:
        try:
            owner = CustomUser.objects.get(id=owner_id)
        except CustomUser.DoesNotExist:
            return Response({'error': 'Owner not found'}, status=404)
        queryset = TaskEventGroup.objects.filter(owner=owner)
    event_ids = []
    for group in queryset:
        event_ids.append(list(group.events.values_list('id', flat=True)))
    serializer = self.serializer_class(queryset, many=True)
    group_data = serializer.data
    for i, group in enumerate(group_data):
        group['events'] = event_ids[i]
    return Response(group_data, status=200)
```

Figure 4.10 – view all groups endpoint

```python
def retrieve(self, request, pk=None):
    owner_id = request.query_params.get('owner')
    if not owner_id:
        return Response({'error': 'Owner is required'}, status=400)
    try:
        owner = CustomUser.objects.get(id=owner_id)
    except CustomUser.DoesNotExist:
        return Response({'error': 'Owner not found'}, status=404)
    try:
        task_event_group = TaskEventGroup.objects.get(pk=pk)
    except TaskEventGroup.DoesNotExist:
        return Response({'error': 'TaskEventGroup not found'}, status=404)
    if task_event_group.owner != owner:
        return Response({'error': 'Unauthorized'}, status=401)
    event_ids = list(task_event_group.events.values_list('id', flat=True))
    serializer = self.serializer_class(task_event_group)
    group_data = serializer.data
    group_data['events'] = event_ids
    return Response(group_data, status=200)
```

Figure 4.11 – view one group endpoint

```python
def destroy(self, request, pk=None):
    owner_id = request.query_params.get('owner')
    if not owner_id:
        return Response({'error': 'Owner is required'}, status=400)
    try:
        owner = CustomUser.objects.get(id=owner_id)
    except CustomUser.DoesNotExist:
        return Response({'error': 'Owner not found'}, status=404)
    try:
        task_event_group = TaskEventGroup.objects.get(pk=pk)
    except TaskEventGroup.DoesNotExist:
        return Response({'error': 'TaskEventGroup not found'}, status=404)
    if task_event_group.owner != owner:
        return Response({'error': 'Unauthorized'}, status=401)
    task_event_group.delete()
    return Response({'message': 'TaskEventGroup deleted successfully'}, status=204)
```

Figure 4.12 – delete group endpoint

```python
def update(self, request, pk=None):
    owner_id = request.query_params.get('owner')
    if not owner_id:
        return Response({'error': 'Owner is required'}, status=400)
    try:
        owner = CustomUser.objects.get(id=owner_id)
    except CustomUser.DoesNotExist:
        return Response({'error': 'Owner not found'}, status=404)
    try:
        task_event_group = self.queryset.get(pk=pk)
    except TaskEventGroup.DoesNotExist:
        return Response({'error': 'TaskEventGroup not found'}, status=404)
    if task_event_group.owner != owner:
        return Response({'error': 'Unauthorized'}, status=401)
    serializer = self.serializer_class(task_event_group, data=request.data, partial=True)
    if serializer.is_valid():
        serializer.save()
        event_ids = list(task_event_group.events.values_list('id', flat=True))
        group_data = serializer.data
        group_data['events'] = event_ids
        return Response(group_data, status=200)
```

Figure 4.13 – update group endpoint

- **PostgreSQL Integration:**
  A PostgreSQL database was used to persist user, event, and group data. The Django ORM was used to define and manage models, including relationships between tasks, groups, and users. See Appendix 1 for full database Entity Relationship Diagram.

Frontend Development (React + Vite)

The frontend was built with React, bundled using Vite for faster development and hot reloading. The interface was designed to be minimalistic and intuitive, focusing on usability for both technical and non-technical users.

- Authentication UI:
  Login and registration forms were created to allow users to authenticate using token-based authentication. Upon successful login, tokens were stored securely for session-based API requests. The registration form is validated using YUP for validation to ensure the information the user enters is correct.

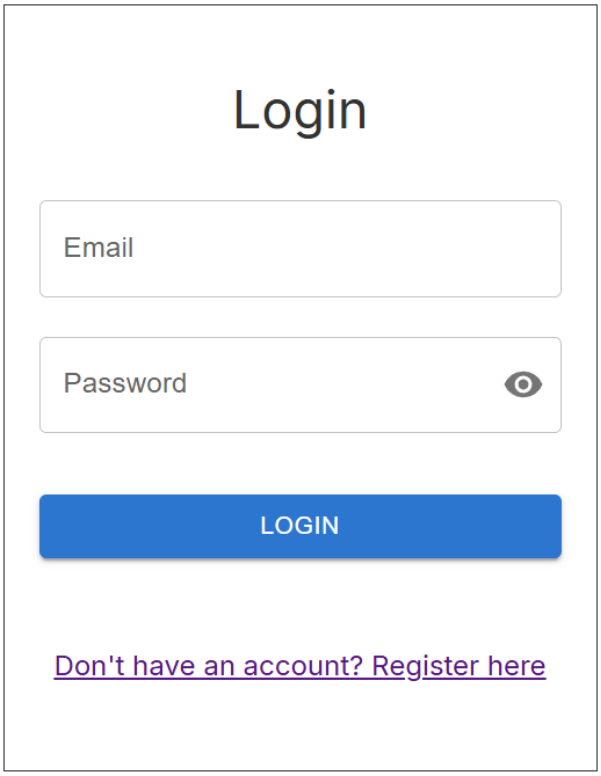


Figure 4.14 – Login form

```
return(
    <div className={"myBackground"}>
        {showMessage ? <MyMessage text={"Login failed. Please try again."} color={"red"}/> : null}
        <form onSubmit={handleSubmit(submission)}>
            <Box className={"whiteBox"}>
                <Box className={"itemBox"}>
                    <Box className={"title"}>Login</Box>
                </Box>
                <Box className={"itemBox"}>
                    <MyTextField label={"Email"} name={"email"} control={control}/>
                </Box>
                <Box className={"itemBox"}>
                    <MyPassField label={"Password"} name={"password"} control={control}/>
                </Box>
                <Box className={"itemBox"}>
                    <MyButton label={"Login"} type={"submit"}/>
                </Box>
                <Box className={"itemBox"} sx={{flexDirection:'column'}}>
                    <Link to="/register">Don't have an account? Register here</Link>
                </Box>
            </Box>
        </form>
    </div>
)
```

Figure 4.15 – Login form code

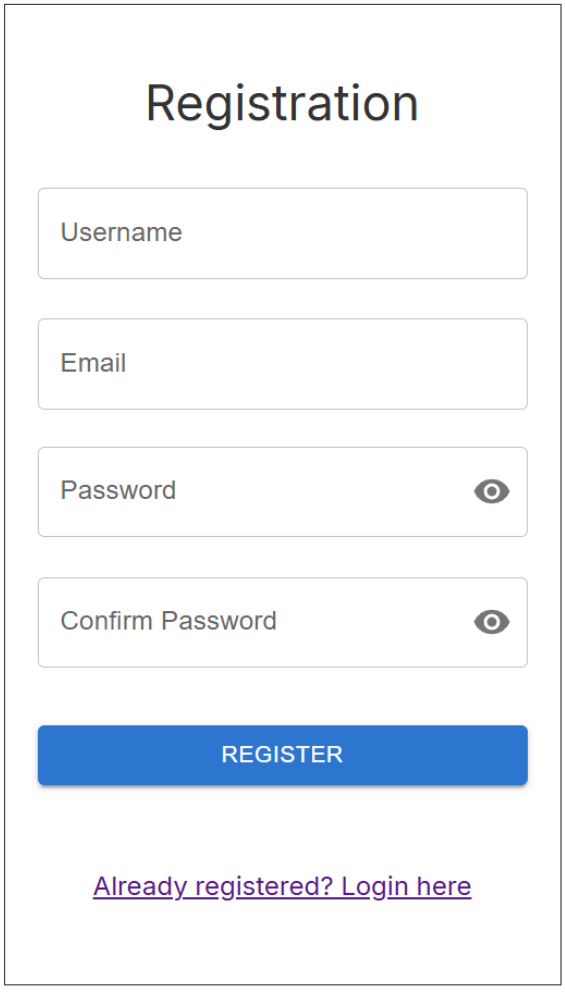


Figure 4.16 – Register form

```
return(
    <div className={"myBackground"}>

        <form onSubmit={handleSubmit(submission)}>
            <Box className={"whiteBox"}>
                <Box className={"itemBox"}>
                    <Box className={"title"}>Registration</Box>
                </Box>
                <Box className={"itemBox"}>
                    <MyTextField label={"Username"} name={"username"} control={control}/>
                </Box>
                <Box className={"itemBox"}>
                    <MyTextField label={"Email"} name={"email"} control={control}/>
                </Box>
                <Box className={"itemBox"}>
                    <MyPassField label={"Password"}  name={"password"} control={control}/>
                </Box>
                <Box className={"itemBox"}>
                    <MyPassField label={"Confirm Password"} name={"password2"} control={control}/>
                </Box>
                <Box className={"itemBox"}>
                    <MyButton type={"submit"} label={"Register"}/>
                </Box>
                <Box className={"itemBox"}>
                    <Link to="/" className={"myLink"}>Already registered? Login here</Link>
                </Box>
            </Box>
        </form>
    </div>
)
```

Figure 4.17 – Register form code

```
const schema = yup.object({
    username: yup.string()
        .required('Username is required')
        .min(4, 'Username must be at least 4 characters long')
        .max(150, 'Username cannot be more than 150 characters long'),
    email: yup.string()
        .required('Email is required')
        .email('Not a valid email address'),
    password: yup.string()
        .required('Password is required')
        .min(8, 'Password must be at least 8 characters long')
        .matches(/[a-z]/, 'Password must contain at least one lowercase letter')
        .matches(/[A-Z]/, 'Password must contain at least one uppercase letter')
        .matches(/[0-9]/, 'Password must contain at least one number')
        .matches(/[^a-zA-Z0-9]/, 'Password must contain at least one special character'),
    password2: yup.string()
        .required('Confirm Password required')
        .oneOf([yup.ref('password'), null], 'Passwords do not match')
})
```

Figure 4.18 – Register form validation schema using YUP form validation

- **Task/Event Management:**
  The event management interface allows users to create, view, update, and delete events. The interface was built using React components and designed to provide a streamlined user experience.

  Each event can include a name, description, start and end datetime, and an optional group assignment. A set of reusable components handle event forms, validation, and form submission via Axios and appropriate HTTP methods (POST, PUT, GET, DELETE) to the backend API. Upon submission, events are stored in the PostgreSQL database and retrieved dynamically using authenticated API calls.

  Events are displayed in a list and calendar-style layout, with a feature for filtering by group. Users can quickly edit or delete entries via embedded controls, and changes are reflected in real time.

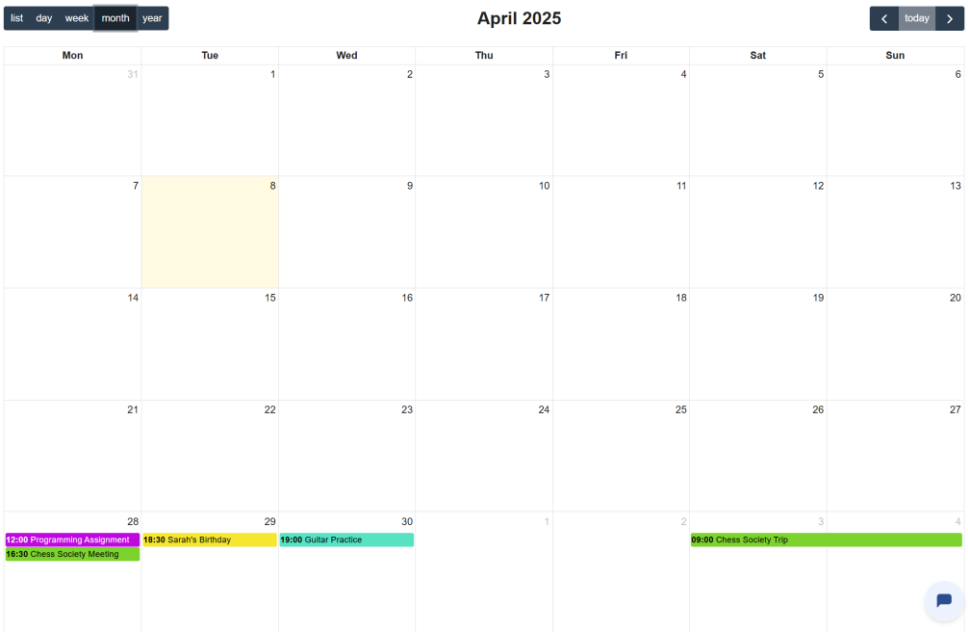


Figure 4.19 – calendar Month view

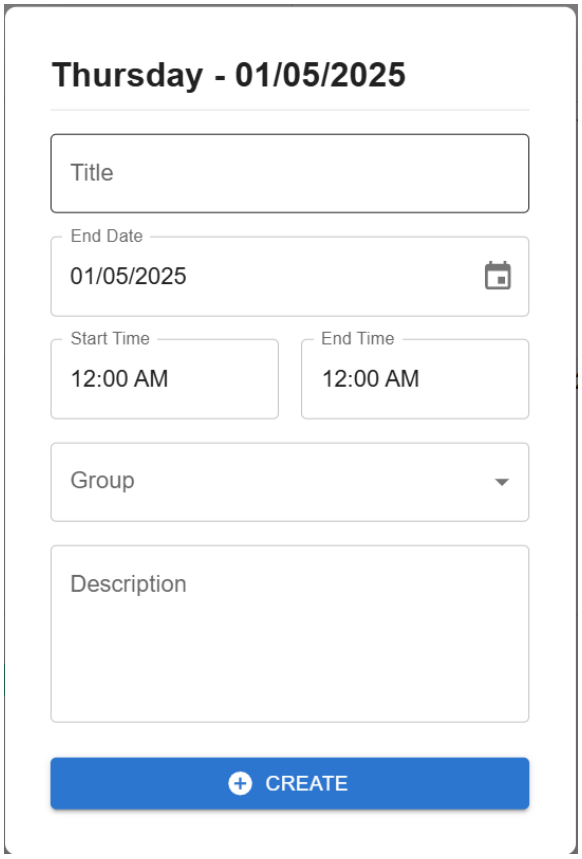


Figure 4.20 – create event modal

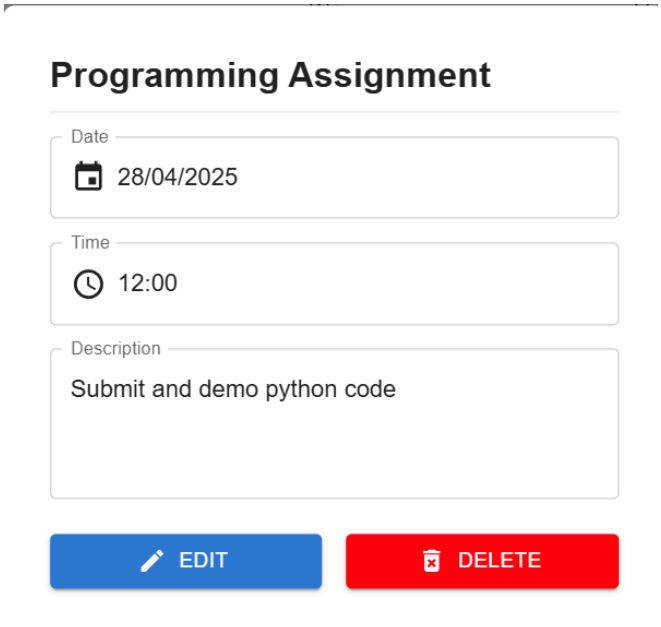


Figure 4.21 – view event details

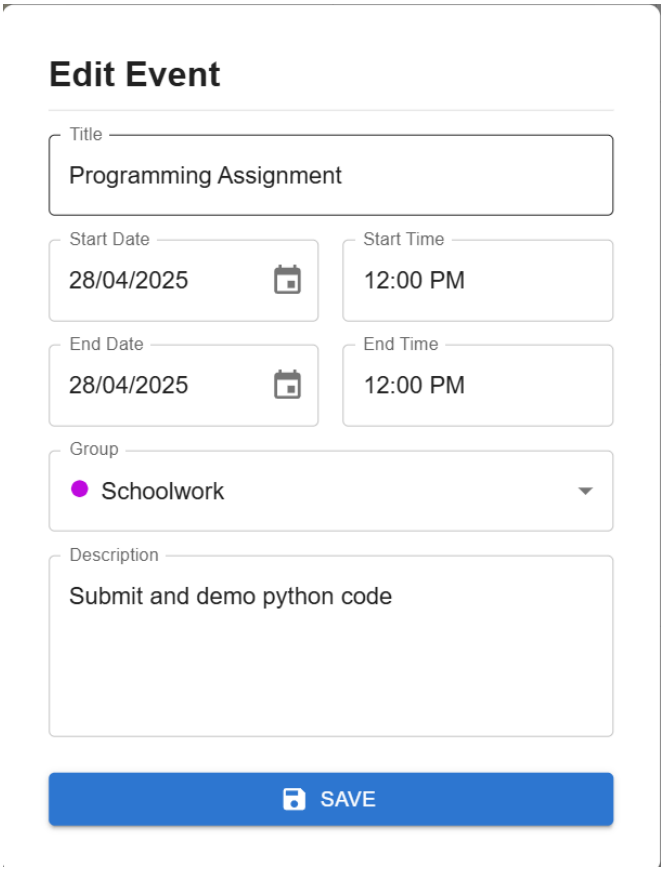


Figure 4.22 – Edit event modal

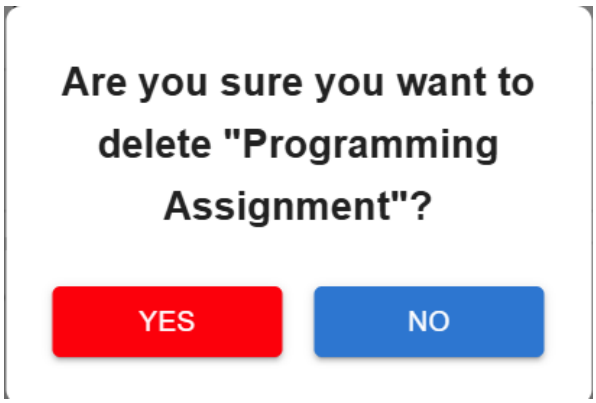


Figure 4.22 – confirm deletion pop-up

- **Groups Management**
  A dedicated interface was developed to allow users to manage their task/event groups, helping them organize their calendar items more effectively. Users could create, edit, and delete groups, each of which included a name, description, and color label for easy visual categorization.

  The group management UI was implemented using React forms and modals. When a group is created or modified, the frontend sends the corresponding API request to the backend using Axios. The selected group can then be assigned to tasks or events during their creation or update process. A color-coded system was used to visually differentiate groups in the task/event listing view.

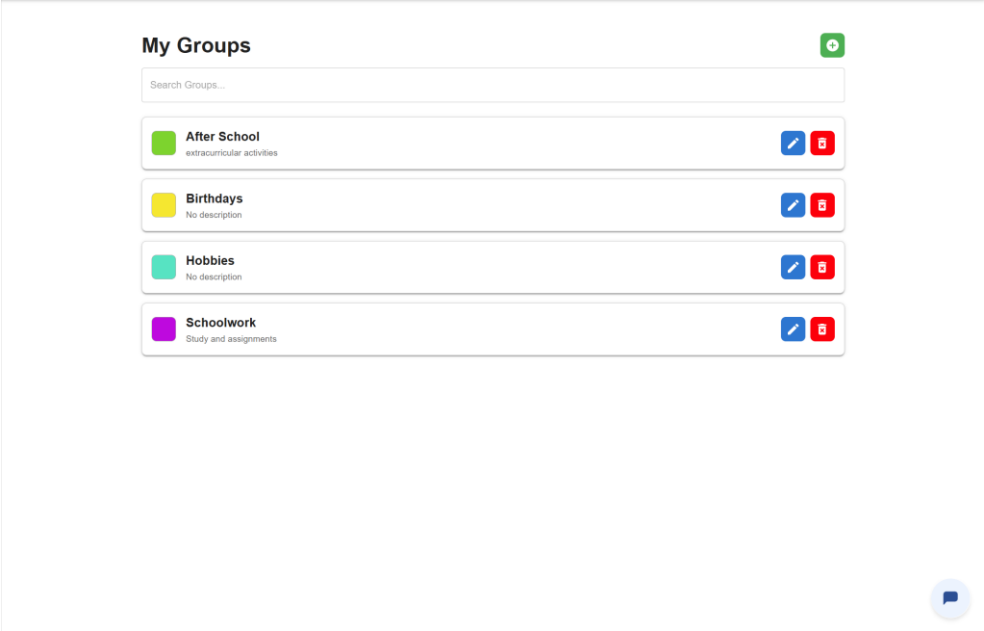


Figure 4.23 – Groups menu

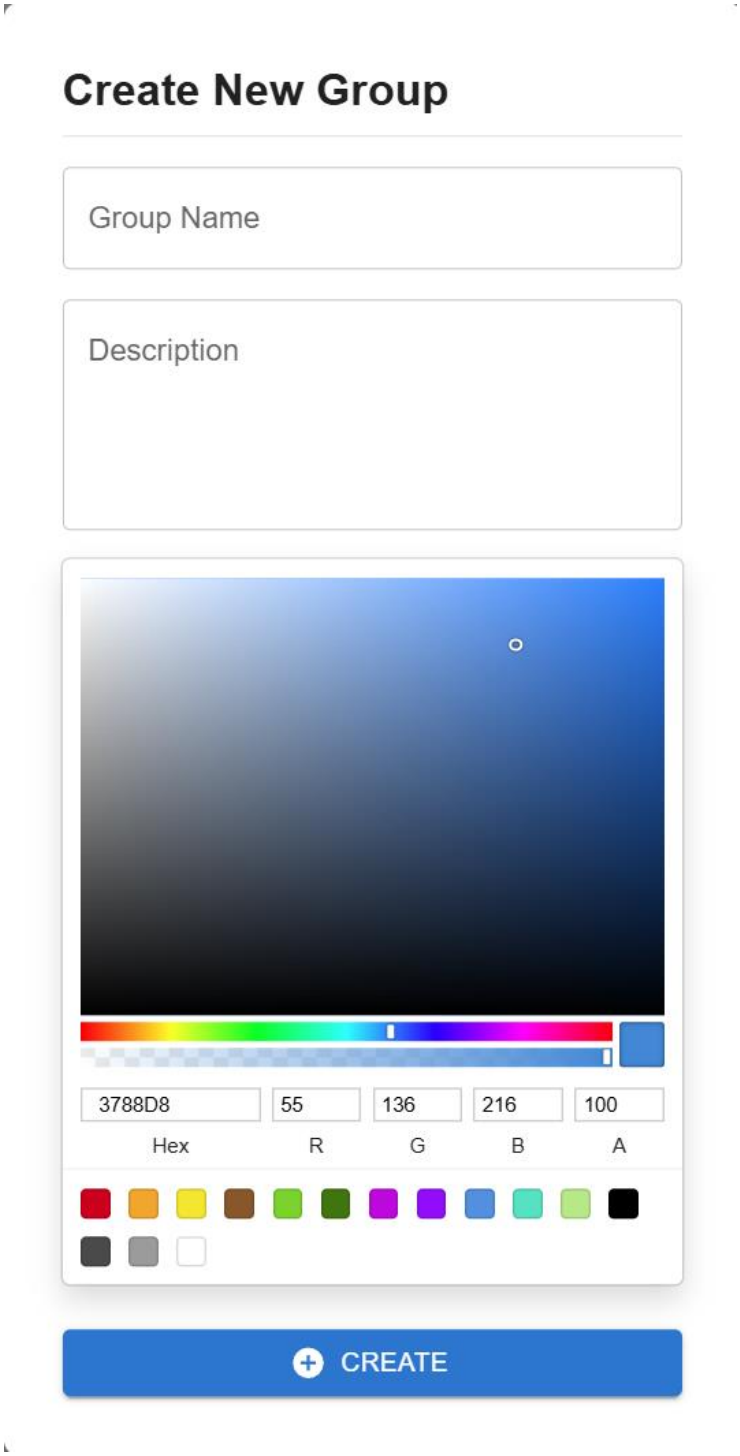


Figure 4.23 – Create group modal

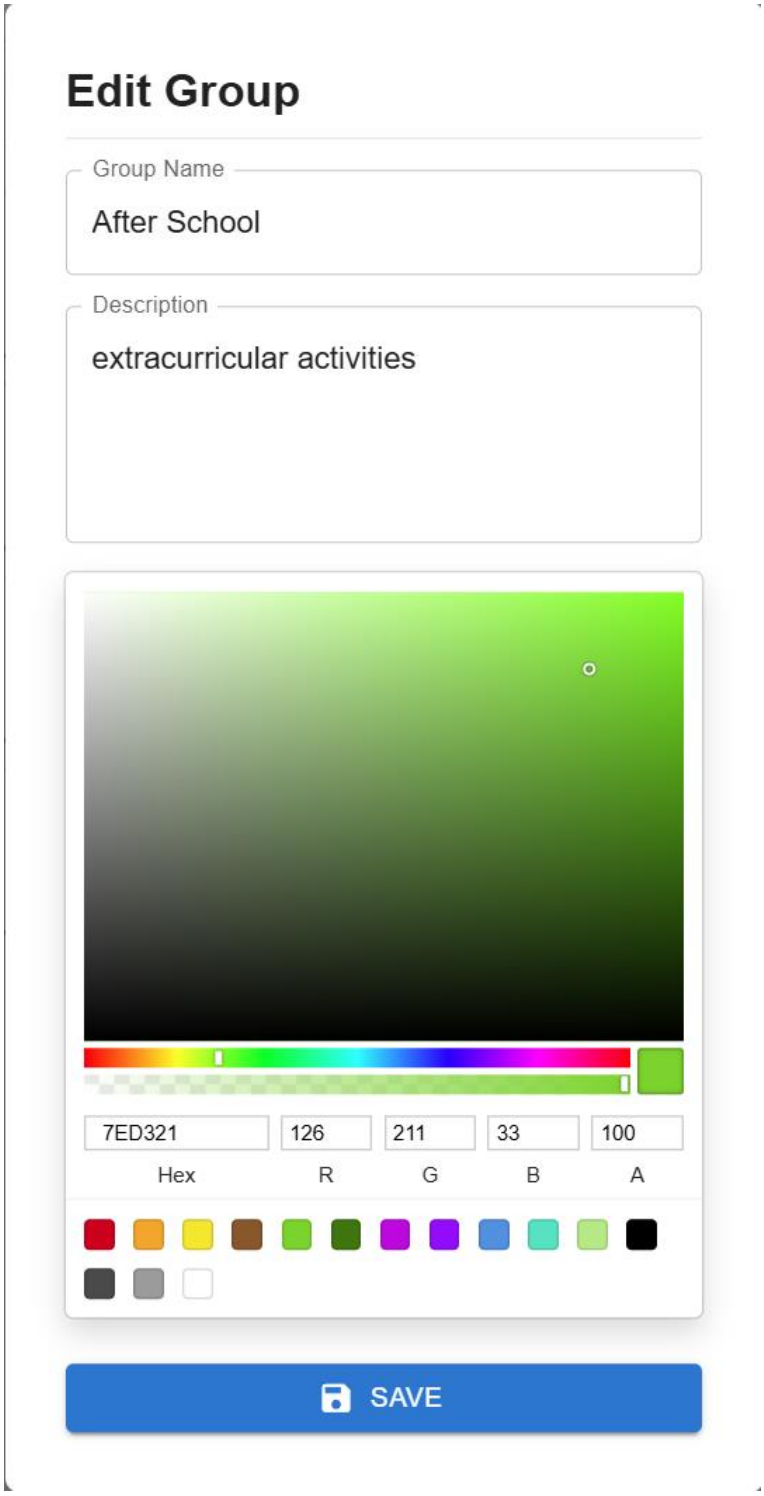


Figure 4.24 – Update group modal

A dropdown menu component displays all existing groups, each represented with its assigned color label. When a user selects a group, the application filters the event list or calendar view to show only the events linked to that group. This filtering is performed client-side based on the data retrieved from the backend.

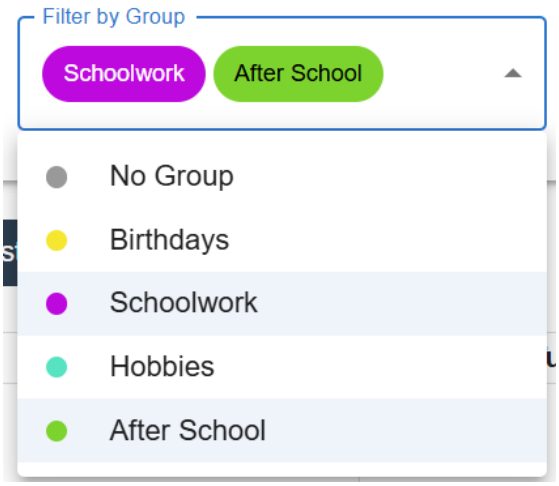


Figure 4.25 – Group filter

- Axios Integration:
  Axios was used for sending HTTP requests to the Django backend. Two Axios instances were created – one for handling event API calls, and the other for handling user API calls.

```
const AxiosCalendarInstance = axios.create({
    baseURL: API_BASE_URL,
    timeout: 5000,
    headers: {
        "Content-Type": "application/json",
        accept: "application/json",

    }
})

export default AxiosCalendarInstance;
```

Figure 4.26 – Axios Calendar Instance

```
const AxiosUserInstance = axios.create({
    baseURL: API_BASE_URL,
    timeout: 5000,
    headers: {
        "Content-Type": "application/json",
        accept: "application/json",

    }
})

AxiosUserInstance.interceptors.request.use(
    (config) => {
        const token = localStorage.getItem('Token')

        if(token){
            config.headers.Authorization = `Token ${token}`
        }
        else{
            config.headers.Authorization = ``
        }
        return config
    }
)

AxiosUserInstance.interceptors.response.use(
    (response) => {
        return response
    },
    (error) => {
        if(error.response && error.response.status === 401){
            localStorage.removeItem('Token')
        }
    }
)

export default AxiosUserInstance;
```

Figure 4.27 – Axios User Instance

Chatbot Development (Botpress + GPT)

The chatbot assistant was developed using **Botpress**, which serves as the primary conversational layer between users and the backend.

- **Intent Recognition and Flow Design:**
  Botpress flows were configured to handle typical user intents such as “Create a task”, “Show my events for today”, or “Delete my meeting”. NLP modules in Botpress helped extract entities like dates, times, and task names.

- **API Calls via Chatbot:**
  When an intent was recognized, Botpress triggered API calls to the backend using dynamic user input. These flows were tested using Botpress’s emulator before integration.

- **GPT Integration:**
  To improve the chatbot’s understanding of natural language, the GPT API was used in specific flows for intent parsing and summarization. This allowed the assistant to handle more ambiguous phrasing and improve response quality.

- **Chatbot Workflow**
  The workflow starts my retrieving the ID of the logged in user and then fetching their events and groups – stored in their own variables. The workflow then holds at a conversational node which can answer the user’s questions about their events and groups. If the chatbot detects that the user want’s top perform an action that requires an API call, the workflow transitions to the next node which captures the relevant information from the user and saves them to variables. These variables are then used to create a payload which is sent to the relevant API using a HTML method (POST, PUT, DELETE). The API response is captured into a variable and is then used to generate a human readable response which is then shown to the user.

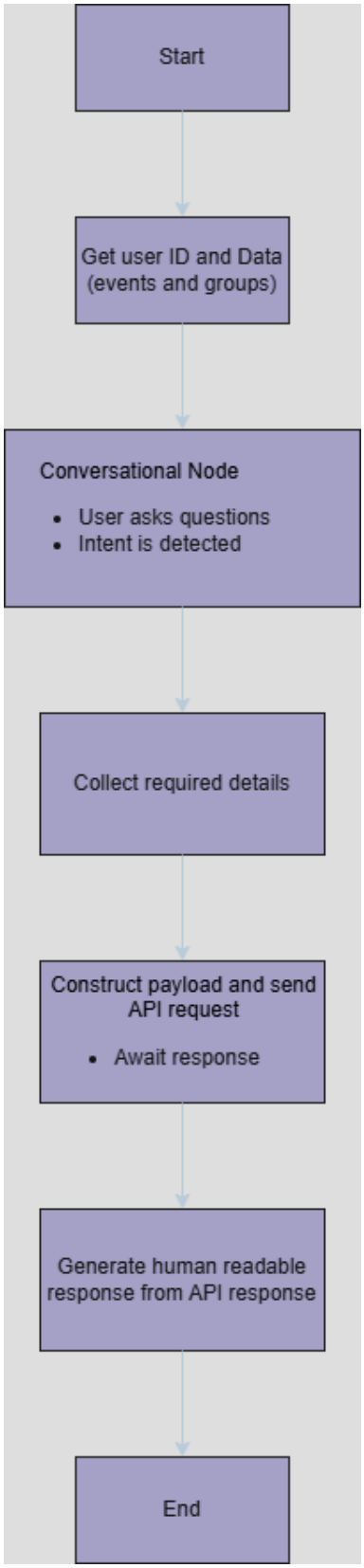


Figure 4.28 – Generalize chatbot flow diagram. See Appendix 2 for complete workflow

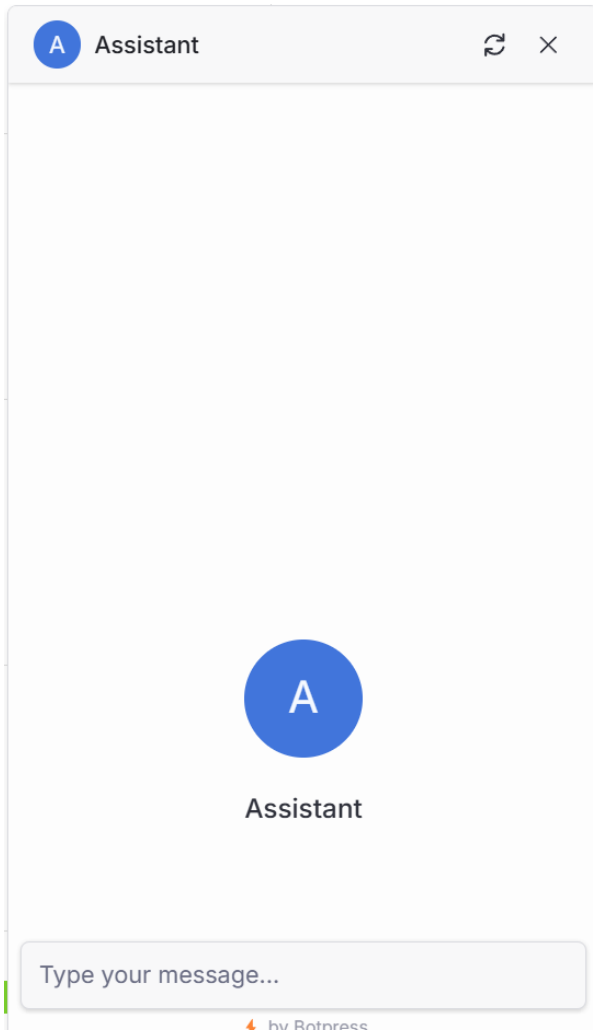


Figure 4.29 – Chatbot pop-up

Hosting and Deployment

The application was deployed using Render, a cloud platform that supports full-stack web application hosting with minimal configuration. Render was chosen for its simplicity, free tier, and ease of integrating both frontend and backend services.

The Django backend was deployed as a web service using a Git-connected Render project. Environment variables, such as database credentials and allowed hosts, were managed through Render's built-in environment settings. PostgreSQL was also set up through Render as a managed database instance, allowing integration with the Django ORM.

For the frontend, a separate static site was created on Render. The React application was built using Vite and deployed by configuring the build command and output directory within Render's static site settings. The frontend was then connected to the backend via environment-configured API URLs.

4.3. Conclusions

The software development process for this project delivered a functional calendar and task management application with integrated AI chatbot capabilities. Each major component—frontend, backend, database, and chatbot—was implemented according to the system design and requirements.

The React-based frontend provides a responsive and modular interface for user interaction, supporting both manual input and natural language communication via the chatbot. On the backend, Django enabled the development of secure APIs, data handling logic, and authentication mechanisms. PostgreSQL served as a robust data store, ensuring reliable performance and support for multiple users and user-specific data segregation.

The chatbot, implemented with Botpress and enhanced through GPT-powered natural language processing, acted as the system's most innovative feature. It demonstrated the ability to assist users in managing their schedules by interpreting natural language commands and executing the appropriate operations through backend API calls. While refinement is still required in terms of flexibility and advanced features, the core chatbot functionalities were implemented.

5. Testing and Evaluation

5.1. Introduction

This chapter outlines the strategies and tools used to test the system throughout its development, as well as the methods used to evaluate the final application. Both automated testing and user-based evaluation were conducted to ensure that the application functioned correctly and that its usability aligned with the goals of the project. The testing focused on validating the technical correctness of the system, while the evaluation focused on how users interacted with both the manual interface and the chatbot assistant.

5.2. System Testing

Testing was conducted on multiple levels to verify the correctness and reliability of the application. Both the backend and frontend components were subject to automated and manual testing processes.

Backend Testing

The backend API was tested using the Python unittest library. These tests were designed to cover a wide range of scenarios, including both valid and invalid API requests. Tests included cases such as:

- Successful user login
- Creation, retrieval, update, and deletion of tasks and events
- Attempted operations with missing fields, incorrect data types, or unauthorized access

These tests were integrated into a GitHub Actions CI pipeline, where they were executed automatically on every push to the main repository. This ensured that the backend remained stable, and regressions were caught early. See Appendix 3 for API test example.

Frontend Testing

Frontend testing was conducted using the Robot Framework, configured to run headless Chrome instances. The tests simulated common user interactions with the web interface, including:

- Logging in and logging out
- Creating, reading, updating, and deleting events and groups
- Navigating through the dashboard and interacting with form elements

Like the backend, these tests were also scheduled to run in GitHub Actions, providing continuous feedback on frontend stability and helping to catch integration errors early. See Appendices 4 and 5 for Robot Test example.

Component-Level Testing

In addition to automated testing, individual components for both the frontend and backend were manually tested during development. As each module was built—such as user authentication, API endpoints, and chatbot interactions—it was tested in isolation before being integrated into the wider system. This modular testing approach allowed issues to be identified and resolved before they affected other parts of the application.

5.3. System Evaluation

To evaluate the usability and effectiveness of the application, user testing sessions were conducted with both tech-oriented and non-tech-oriented participants. Users were asked to complete a series of tasks using the manual UI as well as the AI chatbot.

Tasks included:

- Logging into the application
- Creating and editing a calendar event
- Deleting a task
- Creating and editing a group
- Deleting a group
- Asking the chatbot to summarise upcoming events
- Creating a task using natural language

The evaluation provided valuable insights into the strengths and limitations of the system. Tech-oriented users generally performed slightly better, especially when using the chatbot, likely due to their familiarity with digital systems and natural language prompts. However, all users were able to complete the tasks with minimal guidance, which suggested that the core functionality of the application was intuitive and accessible.

Feedback on the chatbot assistant was generally positive, with users describing it as a “good first iteration” or “a solid proof of concept.” While functional, several users noted that the chatbot responses could be more natural and precise, particularly when interpreting vague or ambiguous prompts. Another point to note, was that the chatbot is not very flexible in its operation – if the users followed to flow logic exactly it works well, but straying from this logic can lead to unexpected behaviours. This highlighted a need for further refinement of the natural language understanding and intent recognition modules.

See Appendix 6 for example of user evaluation notes.

5.4. Conclusions

The testing phase demonstrated that the system was stable, functional, and capable of supporting key user interactions across both manual and chatbot interfaces. Automated tests, running through continuous integration pipelines, helped maintain code reliability during development. Component-level testing ensured that each part of the application worked as expected before being combined.

The user evaluation confirmed that the application's core features were usable and met the intended goals. Although the chatbot assistant requires further refinement to improve its conversational accuracy and flexibility, it served as an effective proof of concept and a solid foundation for future development.

6. Conclusions and Future Work

6.1. Introduction

This chapter presents a summary of the work carried out during the development of the calendar and task management application with an integrated AI chatbot assistant. It begins with an overview of the project's outcomes and reflects on how well the original goals and objectives were achieved. This is followed by a discussion of potential future enhancements to the system, including improvements to chatbot functionality and extended feature sets. Finally, the chapter concludes with a brief reflection on the personal learning journey undertaken throughout the project.

6.2. Goals Assessment

At the start of this project, the primary goal was to develop a web-based calendar and task management application with an integrated conversational AI chatbot assistant. The system aimed to offer users a hybrid approach to managing their schedules—through both a traditional graphical user interface and a natural language interface through a chatbot.

The following key objectives were outlined at the start of the project, and their progress is assessed below:

- **Implement a full-stack application with user authentication**
  Achieved. A secure authentication system was implemented using Django's built-in user management features, with hashed passwords and token-based session handling via Knox.
- **Allow users to create, view, update, and delete tasks/events**
  Achieved. Full CRUD functionality was provided for both tasks and events, accessible through a responsive React frontend and protected backend API endpoints.
- **Integrate an AI chatbot assistant capable of performing CRUD operations**
  Partially Achieved. The chatbot, developed using Botpress and integrated with OpenAI's GPT model, is capable of understanding user commands and performing the relevant operations by calling backend APIs. However, it's understanding of more nuanced or ambiguous language was limited. Further refinement would be required to improve its conversational flexibility and error handling.
- **Store and organize tasks/events using groups**
  Achieved. Users can assign events to color-coded groups for improved organization and filtering within the interface.
- **Design with usability and data security in mind**
  Achieved. The interface was designed to be clean and intuitive, and security measures such as password hashing, data isolation per user, and secure API endpoints were implemented.

In summary, the project successfully met most of its original goals. While the core system is fully functional, there is still room for improvement in areas such as chatbot accuracy and the expansion of advanced features.

6.3. Future Work

While the core features of the application were implemented, there are several areas identified for improvement and expansion. These enhancements would increase usability, accessibility, customization, and the robustness of the system.

User Account and Profile Enhancements

- Implementation of a dedicated profile page allowing users to upload a profile picture.
- Support for users to change their username or password through the interface.
- Addition of password recovery and reset functionality, including email-based reset links for forgotten passwords.

Productivity and Feature Extensions

- Introduction of reminder functionality, such as email notifications for upcoming events and deadlines.
- Support for importing and exporting events, allowing users to back up or transfer their data.
- Ability to create multi-user events, enabling collaborative scheduling for shared activities or meetings.

Conversational Assistant Improvements

- Continued refinement of the chatbot's logic, improving natural language understanding and its ability to handle complex commands or edge cases.
- Addition of voice command functionality, enabling users to interact with the assistant through speech-to-text and receive verbal responses via text-to-speech integration.

Customization and Group Management

- Enhanced grouping and sorting capabilities, such as filtering by date, priority level, or completion status.
- Advanced customization options, including colour themes, dark mode, and layout preferences.

Accessibility and Onboarding

- Implementation of key accessibility features, such as a colourblind-friendly mode and screen reader compatibility, to make the application more inclusive.
- A tutorial for first-time users, offering guided walkthroughs of the app's features and chatbot interaction.

User Experience Enhancements

- Support for Google authentication, allowing users to log in using their Google account for convenience.
- Improved mobile device responsiveness, ensuring the application works seamlessly on smaller screens.
- Addition of icons, image, and location support in events, enabling richer context for scheduled items.

6.4. Personal Journey

At the beginning of this project, I wanted to combine a traditional calendar and task management system with a conversational AI assistant. My initial expectation was that the integration of the chatbot would be simple, and that most of my time would be spent on designing the interface and implementing CRUD operations. However, as the project progressed, I found out that working with natural language inputs, particularly through a framework like Botpress, introduced a few challenges that required more work and problem-solving than expected.

Early in development, I underestimated how complex chatbot flows could become. I expected to map commands to API calls with minimal effort, but realized that properly handling variables, understanding intent, and validating user inputs required careful design and testing. There were also failed implementations along the way—especially during the initial attempts at handling ambiguous user inputs or chaining multi-step commands. Some of these ideas had to be simplified or refactored entirely for the chatbot to remain usable and predictable. Additionally, I wanted to implement email reminders into the application, but the emails would not be received by the user despite the application logs indicating that the email was sent. This feature was postponed and placed in the future work section.

Throughout the project, I developed several technical skills. On the frontend, I learned how to write webpages in React and Vite, learning how to build modular, reusable components and manage API state changes effectively. On the backend, I learned how to use Django and Django REST Framework, including authentication, permissions, and API design. I experienced integration of a PostgreSQL database into a full-stack project and managing relational data models with foreign key relationships. I learned how to create a chatbot workflow in Botpress. Finally, I learned how to host a full-stack application on Render, ensuring that all components are connected correctly and securely.

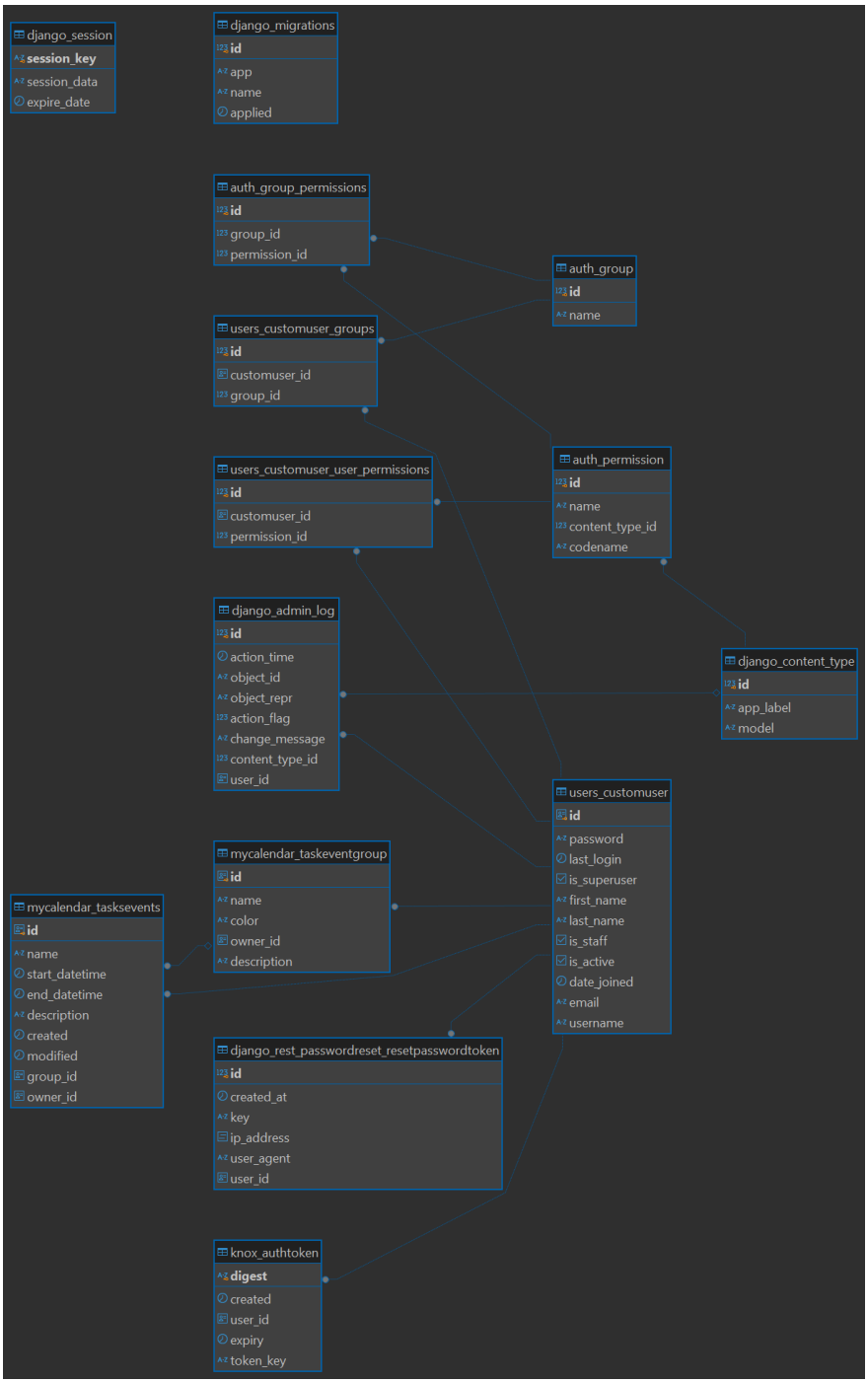
Perhaps most importantly, I learned how to plan, adjust, and iterate in response to feedback and unexpected technical constraints. Working without a strict structure, I had to stay organized, prioritizing development tasks based on what was most stable and functional at each stage.

While not every idea I had at the start made it into the final prototype, I am satisfied with the outcome. The project met its core goals and gave me some insight into real-world development challenges.

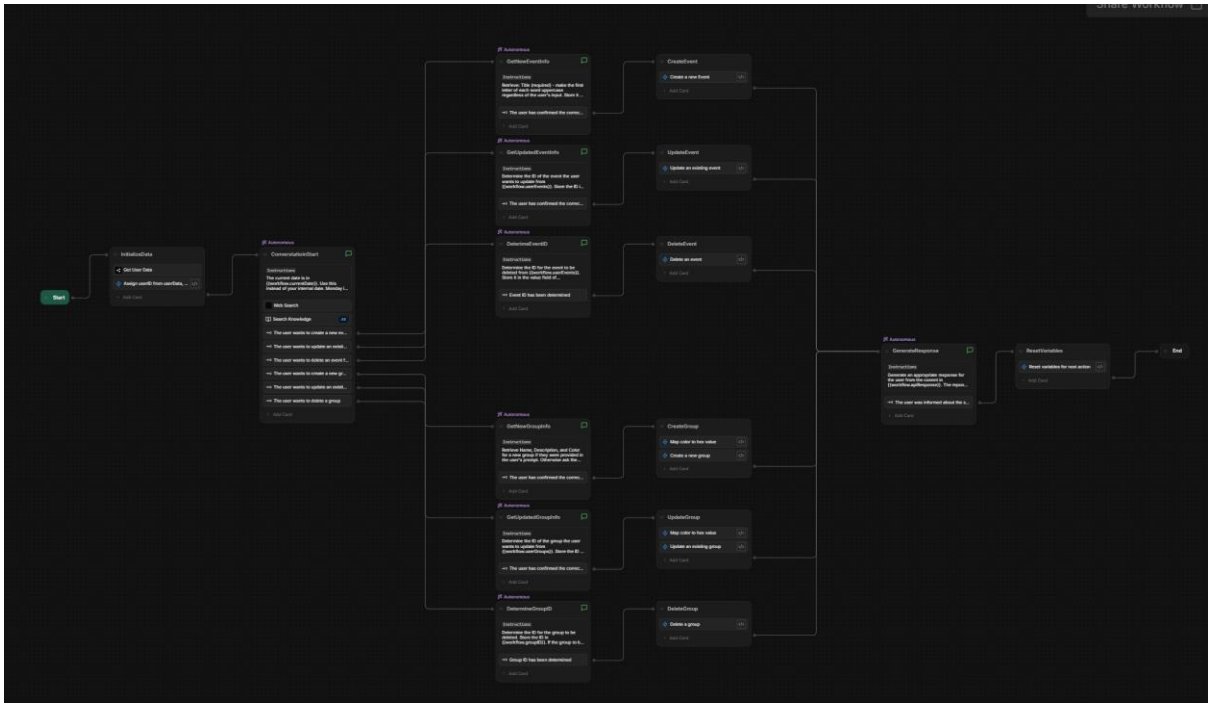
Bibliography

[1] S. Aggarwal, 'Modern Web-Development using ReactJS', vol. 5, no. 1, 2018.
[2] J. Cincovic, S. Delcev, and D. Draskovic, 'Architecture of web applications based on Angular Framework: A Case Study'.
[3] S. Chen, U. R. Thaduri, and V. K. R. Ballamudi, 'Front-End Development in React: An Overview', *Eng. int. (Dhaka)*, vol. 7, no. 2, pp. 117–126, Dec. 2019, doi: 10.18034/ei.v7i2.662.
[4] E. Saks, 'JavaScript frameworks: Angular vs React vs Vue'.
[5] B. Sutar and D. P. Adkar, 'Angular JS and Its Important Component', vol. 2, no. 11, 2019.
[6] D. Ghimire, 'Comparative study on Python web frameworks: Flask and Django'.
[7] V. R. Vyshnavi and A. Malik, 'Efficient Way of Web Development Using Python and Flask', vol. 6, no. 2, 2019.
[8] C. Győrödi, R. Győrödi, G. Pecherle, and A. Olah, 'A comparative study: MongoDB vs. MySQL', in *2015 13th International Conference on Engineering of Modern Electric Systems (EMES)*, Jun. 2015, pp. 1–6. doi: 10.1109/EMES.2015.7158433.
[9] A. Makris, K. Tserpes, G. Spiliopoulos, D. Zissis, and D. Anagnostopoulos, 'MongoDB Vs PostgreSQL: A comparative study on performance aspects', *Geoinformatica*, vol. 25, no. 2, pp. 243–268, Apr. 2021, doi: 10.1007/s10707-020-00407-w.
[10] K. P. Gaffney, M. Prammer, L. Brasfield, D. R. Hipp, D. Kennedy, and J. M. Patel, 'SQLite: past, present, and future', *Proc. VLDB Endow.*, vol. 15, no. 12, pp. 3535–3547, Aug. 2022, doi: 10.14778/3554821.3554842.
[11] OpenAI *et al.*, 'GPT-4 Technical Report', Mar. 04, 2024, *arXiv*: arXiv:2303.08774.Available: http://arxiv.org/abs/2303.08774
[12] K. Roose, 'GPT-4 Is Exciting and Scary', *The New York Times*.
[13] A. Chopra, A. Prashar, and C. Sain, 'Natural Language Processing', 2013.
[14] 'What is GDPR, the EU's new data protection law?', GDPR.eu. Available: https://gdpr.eu/what-is-gdpr/
[15] A. Bhharathee, S. Vemuri, B. Bhavana, and K. Nishitha, 'AI-Powered Student Assistance Chatbot', in *2023 International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*, Jan. 2023, pp. 487–492. doi: 10.1109/IDCIoT56793.2023.10053439.
[16] T. P. Nagarhalli, V. Vaze, and N. K. Rana, 'A Review of Current Trends in the Development of Chatbot Systems', in *2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS)*, Mar. 2020, pp. 706–710. doi: 10.1109/ICACCS48705.2020.9074420.
[17] 'Khanmigo for learners: Always-available tutor, powered by AI'.. Available: https://khanmigo.ai/learners

Appendices



Appendix 1 – Full database ERD



Appendix 2 – Complete chatbot workflow

```python
def test_read_all_events(self):
    """Test reading all events"""

    url = f"{API_BASE_URL}/tasksevents/?owner={USER_ID}"
    response = requests.get(url)

    self.assertEqual(response.status_code, 200, "Expected 200 OK")

    data = response.json()
    self.assertIsInstance(data, list, "Expected response to be a list")

    for event in data:
        self.assertIn("id", event)
        self.assertIn("title", event)
        self.assertIn("owner", event)
        self.assertIn("start", event)
        self.assertIn("end", event)
        self.assertIn("description", event)
        self.assertIn("group", event)
        self.assertEqual(event["owner"], USER_ID, "Event owner mismatch")
```

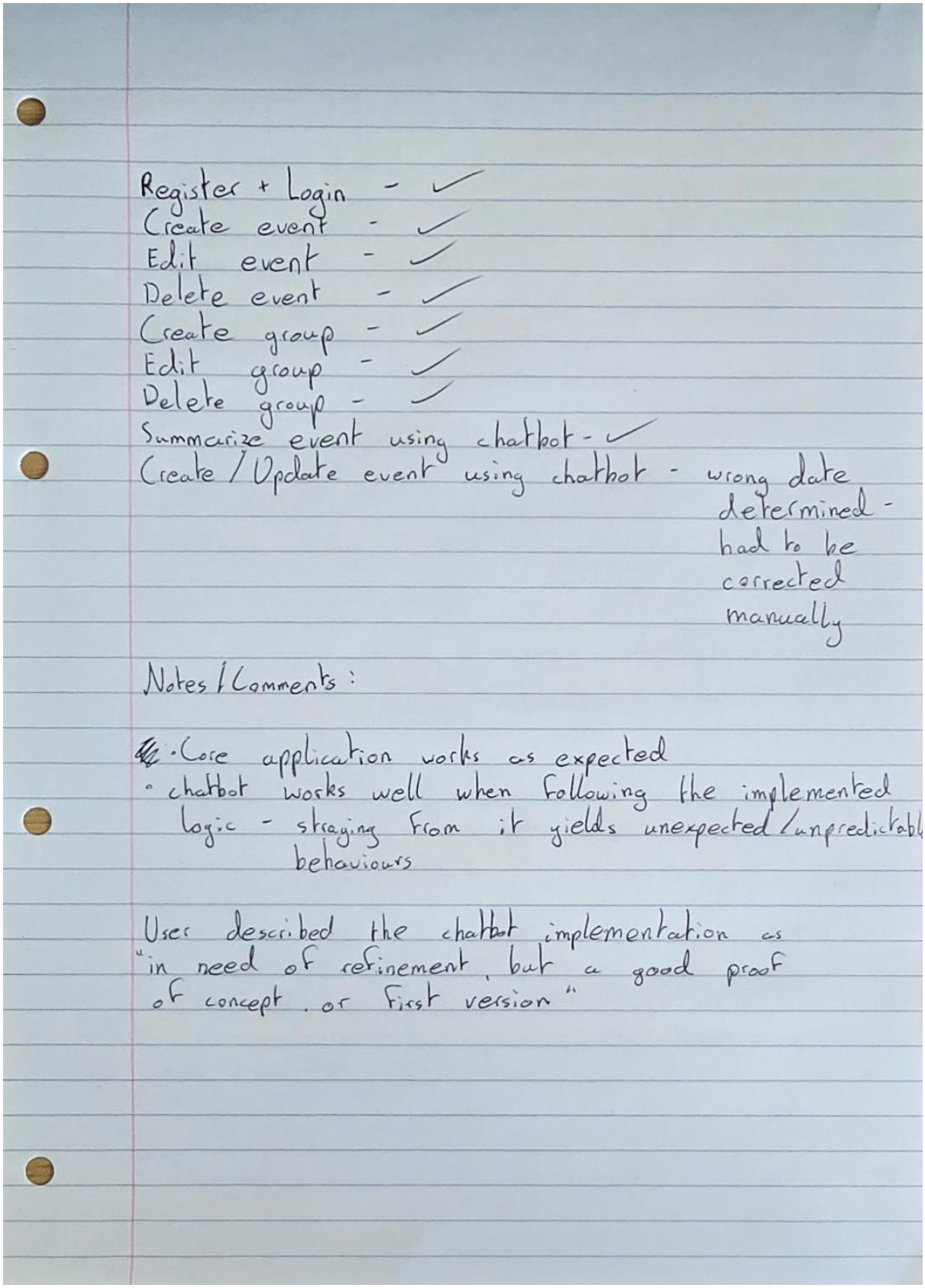
Appendix 3 – API test to fetch all events belonging to a user

```
Event Functions Test
    [Tags]    events
    [Documentation]    This test case verifies the event CRUD functionality of the application.
    Open Login Page
    Login
    Select view    ${MONTH_VIEW_SELECTOR_XPATH}
    ${date}    Get Current Date    result_format=%Y-%m-%d
    Create Event    ${date}    Test Event Title    test group 1    Test Event Description
    Update Event    ${date}    Test Event Title    Updated Event Title
    Delete Event    ${date}    Updated Event Title
    [Teardown]    Close Browser
```

Appendix 4 – Robot Test to test event CRUD functionality

```
Create Event
    [Arguments]    ${date}    ${title}    ${group}    ${description}
    ${DATE_XPATH}    Replace String    ${DATE_CELL_XPATH}    placeholder    ${date}
    Click Element    ${DATE_XPATH}
    Wait Until Element Is Visible    ${EVENT_TITLE_XPATH}    timeout=10s
    Input Text    ${EVENT_TITLE_XPATH}    ${title}
    Click Element    ${EVENT_GROUP_DROPDOWN_XPATH}
    Wait Until Element Is Visible    ${EVENT_GROUP_OPTIONS_XPATH}    timeout=10s
    ${GROUP_OPTION_XPATH}    Replace String    ${EVENT_GROUP_OPTION_XPATH}    placeholder    ${group}
    Click Element    ${GROUP_OPTION_XPATH}
    Input Text    ${EVENT_DESCRIPTION_XPATH}    ${description}
    Click Element    ${CREATE_BUTTON_XPATH}
    Sleep    2s
    Element Should Contain    ${DATE_XPATH}    ${title}
```

Appendix 5 – Create Event custom keyword definition



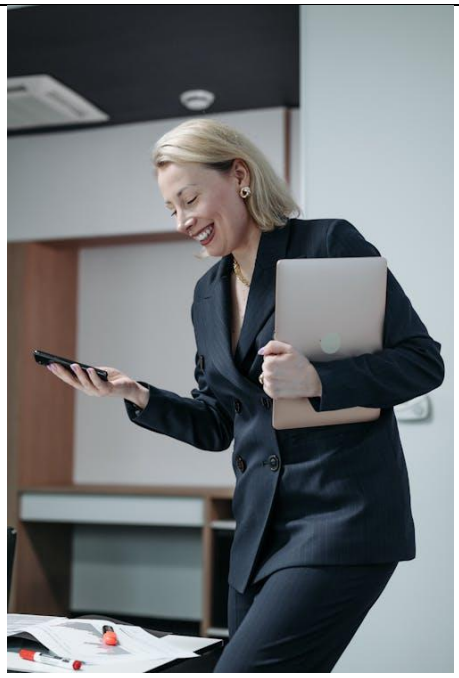
Appendix 6 – user evaluation notes

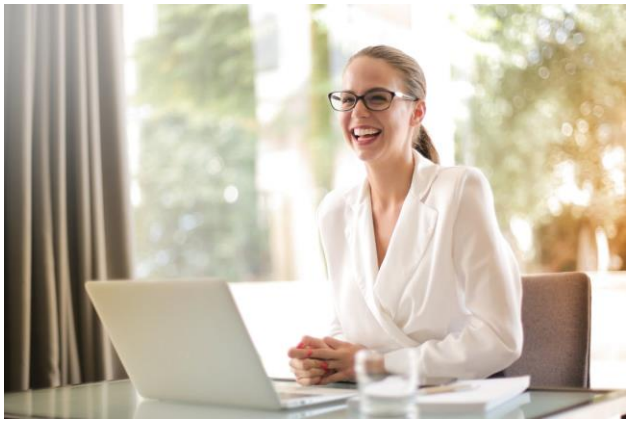
Persona 1: Alex, the Tech Enthusiast

- Background: Alex is a 29-year-old software developer working at a tech startup. He is highly familiar with productivity tools, uses various automation apps, and has a keen interest in exploring the latest in AI and technology.
- Needs and Goals: Alex wants a task management solution that integrates well with his busy work schedule and provides automation capabilities. He values the ability to use voice commands and natural language input to speed up his workflow and reduce the time spent on manual data entry.
- Scenario: Alex logs into the web app to manage his upcoming tasks. While coding, he remembers an important meeting and quickly types a command into the AI chatbot: "Schedule a meeting with the marketing team next Tuesday at 2 PM." The chatbot processes the request and creates the meeting directly in Alex's calendar. Alex also uses the manual interface to view his full weekly schedule in detail, appreciating the seamless integration between the chatbot and the traditional UI.

Persona 2: Linda, the Non-Tech Savvy User

- Background: Linda is a 54-year-old office manager at a small local business. She is comfortable with basic digital tools like email and Microsoft Word but finds new technology challenging to learn and prefers simple interfaces.
- Needs and Goals: Linda wants a straightforward way to keep track of her appointments and manage deadlines without dealing with complex features. She appreciates the ability to use the system without needing to learn new interfaces and values any tool that makes managing tasks easier.
- Scenario: Linda logs into the web app and sees a simple and clear interface for adding



new tasks manually. She uses the chatbot to ask, "Can you remind me about the team meeting tomorrow morning?" The chatbot sets a reminder and confirms it in plain language. Linda later views the reminder in her task list using the manual interface, pleased that she didn't have to navigate through multiple menus to add it.	
Persona 3: Jenny, the Casual User • **Background:** Jenny is a 34-year-old working professional who uses digital tools primarily for personal planning rather than work. She often schedules family events, personal appointments, and recreational activities but doesn't rely heavily on technology beyond her phone calendar and basic reminder apps. • **Needs and Goals:** Jenny wants a simple, reliable way to keep track of personal events like doctor's appointments, gym sessions, and family gatherings. She values convenience and ease of use, preferring a task management app that allows her to quickly add or view events without unnecessary complexity. • **Scenario**: Jenny opens the web app and sees an overview of her upcoming week, which includes her gym sessions and a dentist appointment. She uses the AI chatbot to say, "Remind me to pick up groceries on Saturday afternoon." The chatbot interprets her command, schedules it as a reminder, and confirms it back to her in plain language. Later, she manually adds a family dinner to her calendar, appreciating the flexibility of using both the chatbot for quick tasks and the manual interface for more detail.	

Appendix 7 – personas and scenarios